Machine Learning

From Intuition to Implementation



A textbook for BCA students
Course code NBCA-601

Unit I Introduction · Unit II Supervised Learning · Unit III Ensembles & SVM
Unit IV Unsupervised Learning · Unit V Neural Networks

How to Use This Book

Learning Objectives

Every chapter opens with a box like this one, telling you exactly what you will be able to do by the end.

Key Idea

The one idea to remember. If you remember nothing else, remember this box.

Definition: Term

Formal definitions you may be asked to write in exams.

Example

Worked examples with real numbers, solved step by step.

Common Mistake

Mistakes students commonly make, and how to avoid them.

Try It Yourself

Short Python experiments. Run them, change numbers, and watch what happens.

Exam Tip

Frequently asked exam questions and how to answer them.

Contents

1 Introduction to Machine Learning **1**
- 1.1 Why Do We Need Machine Learning? . . . 1
- 1.2 Artificial Intelligence, Machine Learning and Deep Learning . . . 2
- 1.3 Where Is Machine Learning Used? . . . 4
- 1.4 Basic Terminology . . . 4
- 1.5 Types of Machine Learning Systems . . . 5
  - 1.5.1 Supervised Learning . . . 6
  - 1.5.2 Unsupervised Learning . . . 6
  - 1.5.3 Semi-supervised Learning . . . 7
  - 1.5.4 Reinforcement Learning . . . 7
  - 1.5.5 Batch vs. Online Learning . . . 8
  - 1.5.6 Instance-based vs. Model-based Learning . . . 8
- 1.6 The Machine Learning Workflow . . . 9
- 1.7 Main Challenges of Machine Learning . . . 10
  - 1.7.1 Problems with the Data . . . 10
  - 1.7.2 Problems with the Algorithm . . . 11
  - 1.7.3 Testing and Validating . . . 12
  - 1.7.4 No Free Lunch Theorem . . . 12
- Exam Corner . . . 13

2 Statistical Learning **16**
- 2.1 Introduction to Statistical Learning . . . 16
  - 2.1.1 The Statistical Model . . . 16
  - 2.1.2 Why Estimate f? Prediction and Inference . . . 17
  - 2.1.3 Parametric and Non-parametric Methods . . . 18
- 2.2 Supervised and Unsupervised Learning: The Statistical View . . . 18
  - 2.2.1 Supervised Learning . . . 18
  - 2.2.2 Unsupervised Learning . . . 18
- 2.3 Loss Functions, Training Loss and Test Loss . . . 19
  - 2.3.1 Loss Functions . . . 19
  - 2.3.2 Risk, Training Loss and Test Loss . . . 19
  - 2.3.3 Splitting the Data . . . 21
- 2.4 Trade-offs in Statistical Learning . . . 21
  - 2.4.1 The Bias–Variance Trade-off . . . 21
  - 2.4.2 Other Trade-offs . . . 23
- 2.5 Estimating Risk Statistics . . . 24
  - 2.5.1 Hold-out (Validation-set) Estimate . . . 24
  - 2.5.2 k-Fold Cross-Validation . . . 24
  - 2.5.3 Risk Statistics for Classification . . . 25
- 2.6 Sampling Distribution of an Estimator . . . 26
  - 2.6.1 Properties of Good Estimators . . . 27

2.6.2 The Sample Mean . . . 27
2.6.3 Central Limit Theorem . . . 28
2.6.4 A Biased Estimator: the Variance . . . 28
Exam Corner . . . 30

3 Distance-Based Methods, Decision Trees and Naive Bayes 33
3.1 Distance-Based Methods . . . 33
3.1.1 Common Distance Measures . . . 34
3.1.2 Feature Scaling . . . 35
3.2 Nearest Neighbours (k-NN) . . . 35
3.2.1 The Algorithm . . . 35
3.2.2 Choosing k . . . 37
3.2.3 Weighted k-NN and k-NN Regression . . . 37
3.2.4 Strengths and Weaknesses . . . 38
3.3 Decision Trees . . . 38
3.3.1 Idea and Structure . . . 38
3.3.2 Measures of Impurity . . . 38
3.3.3 Worked Example: Building a Tree with ID3 . . . 39
3.3.4 Algorithm Summary . . . 41
3.3.5 Overfitting and Pruning . . . 42
3.3.6 Strengths and Weaknesses . . . 42
3.4 Naive Bayes . . . 42
3.4.1 Bayes' Theorem . . . 42
3.4.2 The Naive Bayes Classifier . . . 43
3.4.3 Worked Example: Play-Tennis Again . . . 44
3.4.4 Laplace Smoothing . . . 44
3.4.5 Variants of Naive Bayes . . . 44
3.5 Putting It Together: The Three Methods in Python . . . 45
3.6 Comparison of the Three Methods . . . 47
Exam Corner . . . 47

Chapter 1

Introduction to Machine Learning

Learning Objectives

After studying this chapter you will be able to:

- explain the relationship between **AI, Machine Learning and Deep Learning**;
- state and apply Samuel's and Mitchell's definitions of Machine Learning;
- use the vocabulary of ML correctly: *instance, feature, label, training set, model, parameter, hyperparameter*;
- classify ML systems as **supervised, unsupervised, semi-supervised** or **reinforcement**;
- distinguish **batch vs. online** and **instance-based vs. model-based** learning;
- fit a straight-line model by hand using the least-squares formula;
- describe the main **challenges** of ML (data problems, *overfitting*, *underfitting*) and the remedies for each;
- answer 2-, 5- and 10-mark university questions on this unit.

> *Machine learning is the field of study that gives computers the ability to learn without being explicitly programmed.*
>
> — Arthur Samuel, 1959

1.1 Why Do We Need Machine Learning?

Suppose you must write a program that detects *spam e-mail*. The traditional approach is to write rules by hand:

- If the subject contains "FREE" ⇒ spam.
- If the sender is unknown and the body contains "winner" ⇒ spam.
- . . . and hundreds more rules.

This approach has three serious weaknesses.

1. **Long, hard-to-maintain rule lists.** Every new spam trick needs a new rule, and rules start to contradict each other.
2. **No adaptation.** Spammers change "FREE" to "FR33" or "fr.ee" and the rules stop working.

3. **Problems too complex for rules.** No one can write explicit rules to recognise a spoken word or a face in a photo.

A machine-learning program instead studies thousands of e-mails already labelled *spam* or *not spam* and discovers the pattern by itself. When spammers change tactics, we simply retrain it on fresh data.

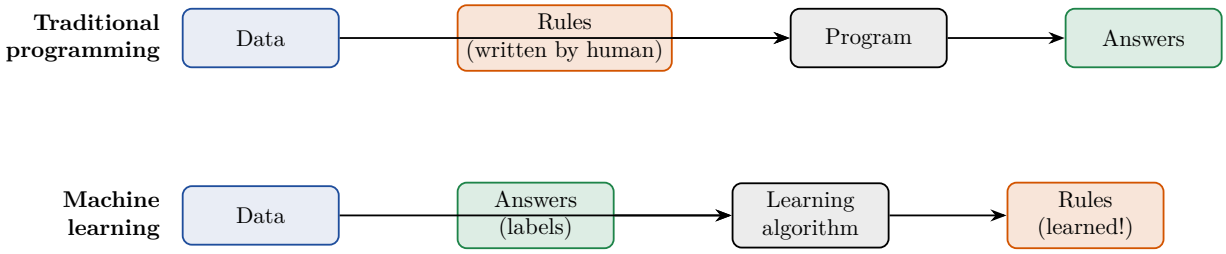


Figure 1.1. Traditional programming versus machine learning.

Reading the Figure

Top row (traditional programming). A programmer writes the *rules*. The program applies those rules to the *data* and produces *answers*.
Bottom row (machine learning). The inputs are swapped. We give the algorithm the *data* together with the correct *answers*, and what comes out is the *rules*. These learned rules can then be applied to new data.
Note the colours: orange marks the *rules*. In the top row a human produces them; in the bottom row the algorithm does. This swap of inputs and outputs is the whole idea of ML.

Key Idea

Traditional programming: *data* + *rules* → *answers*. Machine learning: *data* + *answers* → *rules*.

1.2 Artificial Intelligence, Machine Learning and Deep Learning

Definition: Artificial Intelligence

Artificial Intelligence (AI) is the branch of computer science concerned with building machines that perform tasks that normally require human intelligence, such as reasoning, understanding language, recognising images and making decisions.

Early AI (1950s–1980s) was dominated by **symbolic AI** or *expert systems*, in which experts typed in thousands of "if–then" rules. These systems were useful in narrow areas such as medical diagnosis but broke down on messy real-world data. Since the 1990s the focus has shifted to systems that *learn* from data.

Definition: Machine Learning (Samuel, 1959)

Machine Learning is the field of study that gives computers the ability to learn without being explicitly programmed.

Definition: Machine Learning (Mitchell, 1997)

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

Samuel's definition is informal and historical (he wrote a checkers program that improved by playing against itself). Mitchell's definition is more engineering-oriented because it tells us *what to measure*: to claim that a program is learning we must name the task, the metric and the experience.

Definition: Deep Learning

Deep Learning is a subfield of ML that uses *artificial neural networks with many layers* to learn increasingly abstract representations of data (e.g. pixels → edges → shapes → faces). It is the subject of Unit V.

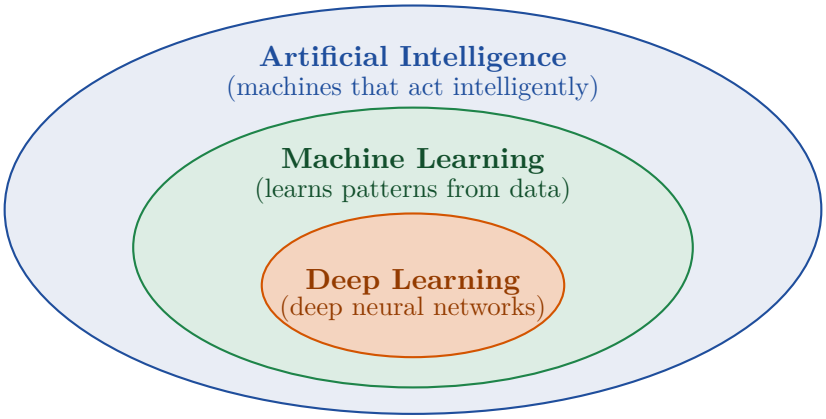


Figure 1.2. AI, ML and Deep Learning as nested sets.

Reading the Figure

The figure is a **set diagram**: each ellipse lies completely inside the one before it.

- *Outer ellipse (AI):* the biggest set. It includes ML but also non-learning AI such as hand-coded chess engines and rule-based expert systems.
- *Middle ellipse (ML):* every ML system is AI, but not every AI system learns.
- *Inner ellipse (DL):* deep learning is one family of ML methods. Decision trees, k-NN and SVMs are ML but *not* deep learning.

Remember the order DL ⊂ ML ⊂ AI.

Example: Spam filter in (T, P, E) form

Task T	Flag new e-mails as spam or not spam
Performance P	Percentage of e-mails classified correctly (accuracy)
Experience E	A collection of e-mails labelled by users

Check: if accuracy rises from 90% to 96% as we add more labelled mail, the program is learning. If accuracy does not change however much data we add, it is *not* learning,

whatever its marketing says.

Example: Which of these is ML?

(a) A program that fixes a date's format using a fixed rule. **Not ML:** no experience is used.

(b) A music app that suggests songs based on what you skipped. **ML:** behaviour (experience) improves recommendations.

(c) A calculator. **Not ML:** its performance never improves with experience.

Exam Tip

"Define Machine Learning" appears almost every year. For full marks write *both* Samuel's and Mitchell's definitions, then give the spam-filter example in (T, P, E) form.

1.3 Where Is Machine Learning Used?

ML is the right tool when (i) rules are too many or too complex, (ii) the environment keeps changing, (iii) the problem has no known algorithmic solution, or (iv) large amounts of data are available to learn from.

Area	Application	ML task
E-mail	Spam filtering	Classification
E-commerce	"Customers also bought"	Recommendation
Health	Detecting disease from X-rays	Image classification
Transport	Self-driving cars	Perception and control
Language	Translation, chatbots	Sequence modelling
Finance	Credit-card fraud detection	Anomaly detection
Real estate	Predicting house prices	Regression
Marketing	Grouping similar customers	Clustering

1.4 Basic Terminology

Before going further, learn this vocabulary. It is used in every chapter.

Term	Meaning
Instance (sample)	One data point, e.g. one e-mail or one house.
Feature (attribute)	An input measurement describing an instance, e.g. area, number of rooms.
Label (target)	The correct output for an instance, e.g. price or "spam".
Training set	Data used to *fit* the model.
Test set	Data held back to *evaluate* the final model on unseen instances.
Model	The function f learned from data, $\hat{y} = f(\boldsymbol{x})$.
Parameters	Numbers learned *by the algorithm* during training (e.g. θ_0, θ_1).
Hyperparameters	Settings chosen *by the human* before training (e.g. k in k-NN, learning rate).

Example: A small dataset

	Area (m^2)	Rooms	Age (yr)	Price (lakh Rs.)
Instance 1	80	2	10	45
Instance 2	120	3	5	78
Instance 3	60	1	20	28

Here *Area, Rooms, Age* are the three **features**; the feature vector of instance 1 is $\boldsymbol{x}_1 = (80, 2, 10)$, and its **label** is $y_1 = 45$. The first three columns together form the input X, and the last column is the target y.

Common Mistake

Do not mix up **parameters** and **hyperparameters**. Parameters are *learned from data*; hyperparameters are *set by you* and are tuned by trying several values and comparing performance on held-out data.

1.5 Types of Machine Learning Systems

ML systems are classified in three independent ways. The same system gets one label from each way, e.g. "a supervised, batch, model-based system".

Q1. How much supervision does it get during training? (supervised, unsupervised, semi-supervised, reinforcement)

Q2. Can it learn incrementally on the fly? (batch vs. online)

Q3. Does it compare new data to stored examples or build a model? (instance-based vs. model-based)

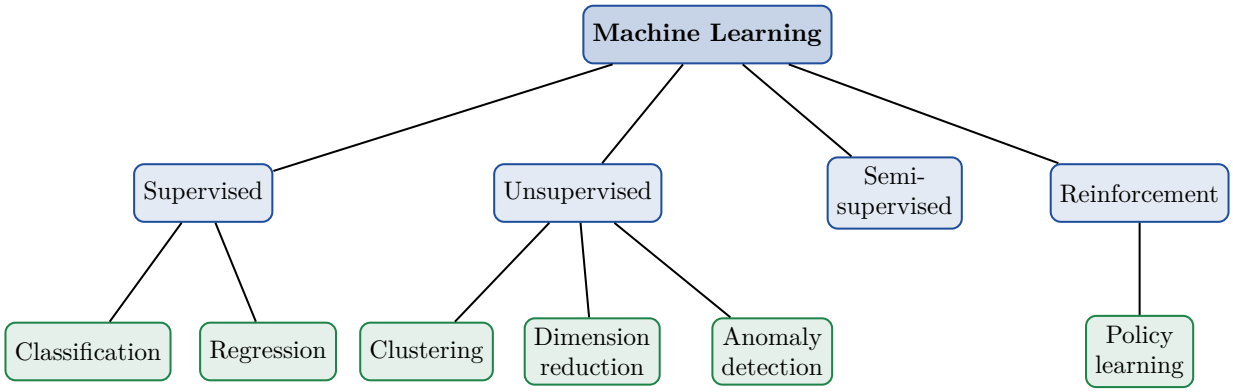


Figure 1.3. Taxonomy of ML systems by the type of supervision. Blue = learning paradigm, green = typical task.

Reading the Figure

Read this **tree from top to bottom**. The root is Machine Learning. The second level (blue) shows the four paradigms; the third level (green) shows the typical *tasks* that each paradigm solves. For example, *classification* and *regression* both need labelled data and therefore sit under *supervised*; *clustering* needs no labels and so sits under *unsupervised*. Semi-supervised learning is a mix of the first two and is usually applied to the same tasks.

1.5.1 Supervised Learning

In supervised learning the training data includes the **correct answer** (the label). The learner is like a student who checks each practice answer against an answer key.

Definition: Supervised Learning

Given a training set $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{n}$ where $\boldsymbol{x}_i \in \mathbb{R}^d$ is the feature vector and y_i is the label, learn a function f such that $f(\boldsymbol{x}) \approx y$ *for new, unseen inputs.*

Types of supervised problems.

- **Classification:** y is a *category.*
  - *Binary:* two classes (spam / not spam).
  - *Multiclass:* one of many classes (digit 0–9).
  - *Multilabel:* several labels at once (a photo tagged "beach" and "sunset").
- **Regression:** y is a *real number* (house price, temperature). If there is one feature it is *simple* regression; if more, *multiple* regression.



Figure 1.4. (a) Classification separates two classes with a decision boundary. (b) Regression fits a line that predicts a number.

Reading the Figure

Panel (a): each point is one instance, plotted by two features x_1, x_2. Blue circles belong to class A and orange triangles to class B. The dashed green line is the *decision boundary* the classifier has learned: points on one side are predicted A, points on the other side B. The *output is a category.*
Panel (b): each dot is a house. The horizontal axis is the feature (size) and the vertical axis is the target (price). The green line is the fitted model; to predict the price of a new house, find its size on the horizontal axis and read the height of the line. The *output is a number.*

1.5.2 Unsupervised Learning

The training data is **unlabelled**. The system must discover structure on its own, like sorting a heap of unfamiliar coins into groups by size and colour without anyone naming them.

Definition: Unsupervised Learning

Given only inputs $\{\boldsymbol{x}_i\}_{i=1}^n$ with no labels, discover hidden structure, such as groups, directions of variation or unusual points, in the data.

- **Clustering:** group similar instances, e.g. segment customers by buying behaviour (K-Means, DBSCAN; Chapter 7).
- **Dimensionality reduction:** compress many features into a few while keeping the information (PCA; Chapter 8). Useful for visualisation and for speeding up learning.
- **Anomaly (outlier) detection:** flag unusual instances, e.g. a suspicious credit-card transaction.
- **Association rule learning:** find relations such as "people who buy bread also buy butter".

1.5.3 Semi-supervised Learning

Labelling is slow and costly (a doctor must label every X-ray), but unlabelled data is cheap. **Semi-supervised** algorithms use a *small labelled* set and a *large unlabelled* set. A typical method is **self-training**: train on the labelled data, predict labels for the unlabelled data, keep the most confident predictions as new labels, and retrain.

Example. Photo apps first cluster faces automatically (unsupervised), then ask you to name each person once (supervised); the name then spreads to every photo in the cluster.

1.5.4 Reinforcement Learning

There are no labelled examples. An **agent** observes the **state** of an **environment**, chooses an **action**, and receives a **reward** (positive) or **penalty** (negative). It learns a **policy**, a rule that says which action to take in each state, that maximises the total reward over time.

Term	Example: robot learning to walk
Agent	The robot's control program
Environment	The floor, gravity, the robot's body
State s	Joint angles, speed, whether upright
Action a	Move a joint motor
Reward r	$+1$ for moving forward, -10 for falling
Policy π	The learned rule: state $\mapsto$ action

Because rewards may arrive late (a chess move is only rewarded when the game ends), the agent maximises the **discounted return**

$$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k\, r_{t+k+1}, \qquad 0 \le \gamma \le 1,$$

where the *discount factor* γ says how much we value future rewards. With $\gamma = 0$ the agent is short-sighted (only the next reward matters); with γ close to 1 it is far-sighted. Famous examples are DeepMind's AlphaGo, which beat the world Go champion, and robots that learn to walk.

Reading the Figure

The picture shows a **closed loop** that repeats at every time step t. (1) The agent looks at the current state and uses its policy to pick an action a_t (top arrow). (2) The environment reacts and sends back the next state s_{t+1} and a reward r_{t+1} (bottom arrow). (3) The agent

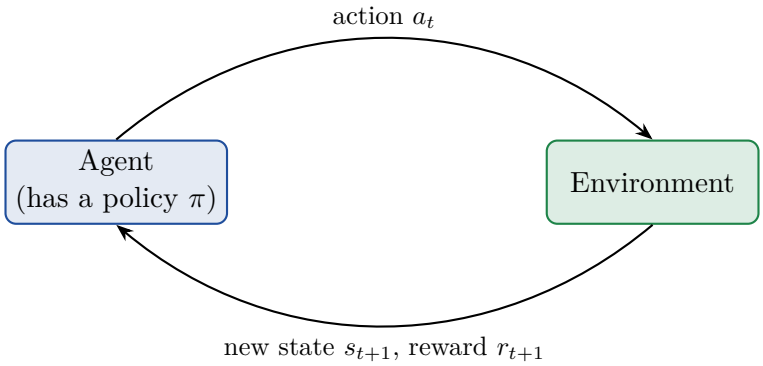


Figure 1.5. The reinforcement-learning loop.

uses that feedback to improve its policy, and the loop repeats. Learning happens by *trial and error*: actions that lead to high reward become more likely.

Type	Labels?	Feedback	Typical task	Example
Supervised	Yes	Correct answer	Classification, regression	Spam, prices
Unsupervised	No	None	Clustering, PCA	Customer groups
Semi-supervised	Few	Partial	Mixed	Photo tagging
Reinforcement	No	Reward/penalty	Sequential decisions	Games, robots

1.5.5 Batch vs. Online Learning

Batch (offline) learning. The system is trained once on *all* available data, then deployed and no longer learns. To include new data it must be retrained from scratch on old + new data. This is simple but can take hours and a lot of memory.

Online (incremental) learning. The system is trained continuously on single instances or small *mini-batches* as they arrive. Each step is cheap, so it suits streaming data (stock prices) and limited memory; it can even handle datasets too large for memory (*out-of-core learning*).

The key hyperparameter of online learning is the **learning rate**: a *high* rate adapts quickly but forgets old data fast and is sensitive to noise; a *low* rate is stable but slow to adapt. A risk is that *bad data* (a faulty sensor) can quickly degrade a live system, so its performance must be monitored.

1.5.6 Instance-based vs. Model-based Learning

Instance-based learning. The system *memorises* the training examples and, for a new point, finds the most *similar* stored ones and copies their answer. *Example:* k-Nearest Neighbours (Chapter 3). No explicit model is built.

Model-based learning. The system builds a *mathematical model* from the examples, then uses only the model (the training data may be thrown away) to predict. *Example:* linear regression.

	Instance-based	**Model-based**
Stores	All training data	Only parameters
Prediction by	Similarity to stored data	Evaluating the learned formula
Training cost	Low (just store)	Higher (fit parameters)
Prediction cost	High (compare to many points)	Low
Example	k-NN	Linear regression

Worked example: a model-based learner by hand

Suppose we believe that house price y depends linearly on size x:

$$\hat{y} = \theta_0 + \theta_1 x.$$

Learning means choosing θ_0, θ_1 so that predictions are close to the data. For simple linear regression the least-squares solution is

$$\theta_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad \theta_0 = \bar{y} - \theta_1 \bar{x}.$$

Example: Fit a line by hand

Data: size x (100 m^2) and price y (lakh Rs.).

x	1	2	3	4	5	$\bar{x} = 3$
y	15	26	29	44	46	$\bar{y} = 32$
$x - \bar{x}$	-2	-1	0	1	2	
$y - \bar{y}$	-17	-6	-3	12	14	
$(x - \bar{x})(y - \bar{y})$	34	6	0	12	28	$\Sigma = 80$
$(x - \bar{x})^2$	4	1	0	1	4	$\Sigma = 10$

Step 1: $\theta_1 = 80/10 = 8$. **Step 2:** $\theta_0 = 32 - 8 \times 3 = 8$.
Model: $\hat{y} = 8 + 8x$. For a new house of size $x = 6$: $\hat{y} = 8 + 8 \times 6 = 56$ lakh Rs..

Try It Yourself

Check the hand calculation with `scikit-learn`. The output should be intercept 8, coefficient 8 and prediction 56.

```
from sklearn.linear_model import LinearRegression

X = [[1], [2], [3], [4], [5]]       # feature: size (100 m^2)
y = [15, 26, 29, 44, 46]            # label: price (lakh Rs)

model = LinearRegression()          # 1. choose the model
model.fit(X, y)                     # 2. learn theta_0 and theta_1
print(model.intercept_, model.coef_[0])   # -> 8.0  8.0
print(model.predict([[6]]))               # -> [56.]
```

Challenge: add the point (6, 58) to the data. Do the parameters change? Why?

1.6 The Machine Learning Workflow

A real ML project is more than choosing an algorithm. It follows a cycle of steps.

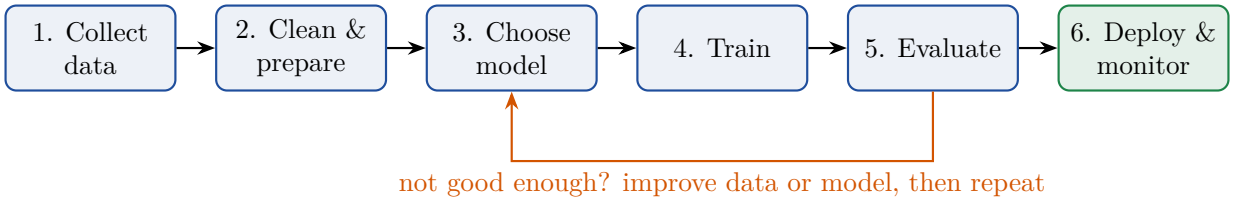


Figure 1.6. The machine-learning workflow.

Reading the Figure

Follow the **blue boxes left to right**; they are the steps in order. The **orange arrow** is the most important part: it shows that evaluation usually sends you *back* to try a different model or better data. ML is therefore iterative.

1. **Collect data:** from databases, sensors, web, surveys.
2. **Clean and prepare:** handle missing values, remove duplicates and outliers, encode text categories as numbers, and *scale* features to comparable ranges. This step often takes most of the project time.
3. **Choose a model** that suits the task (classifier, regressor, clusterer).
4. **Train:** the algorithm finds the parameters that fit the training set.
5. **Evaluate** on data *not* used for training, using the performance measure P.
6. **Deploy and monitor:** use the model on real inputs and watch for performance decay.

1.7 Main Challenges of Machine Learning

Two things can go wrong: *bad data* and *bad algorithm*.

1.7.1 Problems with the Data

1. **Insufficient training data.** A toddler needs to be shown a ball once or twice; ML systems need thousands or millions of examples. Banko and Brill (2001) showed that, given enough data, even simple algorithms perform almost as well as sophisticated ones: *data matters more than the algorithm.*
2. **Non-representative training data.** The training data must represent the cases the model will meet. If you build a "happiness vs. income" model using only rich countries, it will predict poorly for poor countries. This is **sampling bias**. A famous case is the 1936 *Literary Digest* poll: it surveyed 2.4 million people drawn from telephone and car owners, who were mostly wealthy, and wrongly predicted that Landon would beat Roosevelt. A small *sampling noise* (by chance) arises when the sample is too small; *sampling bias* arises when the method of sampling is flawed, even if the sample is large.
3. **Poor-quality data.** Errors, outliers, noise and missing values hide the patterns. Cleaning (discard outliers, fill or drop missing values) is essential.
4. **Irrelevant features.** "Garbage in, garbage out." **Feature engineering** has three parts: *feature selection* (choose the most useful features), *feature extraction* (combine features, e.g. dimensionality reduction) and *creating new features* by collecting new data.

1.7.2 Problems with the Algorithm

Overfitting. The model performs very well on the training data but badly on new data, because it has *memorised the noise* instead of learning the pattern. It is like a student who memorises last year's paper word for word and then cannot answer a new question. Overfitting happens when the model is *too complex* relative to the amount and noisiness of the data.

Underfitting. The model is too simple to capture the structure of the data, so it does badly even on the training set (like a student who read only the chapter titles).

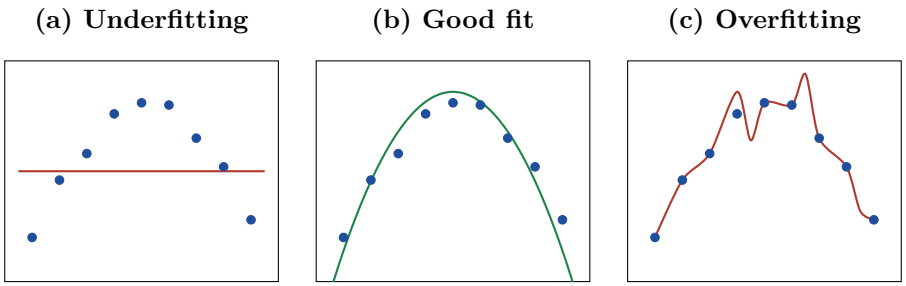


Figure 1.7. The same nine data points fitted by three models of different complexity.

Reading the Figure

All three panels show the *same* blue data points; only the fitted curve (red or green) changes. **(a)** The model is a flat horizontal line. It ignores the rise-and-fall pattern, so errors are large for *every* point: **high training error** ⇒ underfitting. **(b)** A smooth curve passes close to all points without chasing each one: this is the **right complexity** and will generalise well. **(c)** The wiggly curve passes *exactly* through every point, including the random noise. Training error is almost zero, but the zig-zags will mispredict new points: **low training error, high test error** ⇒ overfitting.

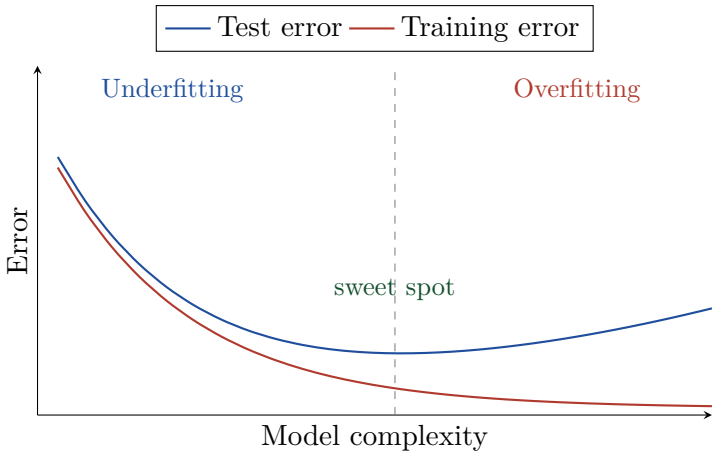


Figure 1.8. Training and test error as model complexity increases.

Reading the Figure

The horizontal axis is how *flexible* the model is (e.g. the degree of a polynomial). The vertical axis is the error.

- **Red curve (training error)** always *falls*: a more flexible model can always fit the training data better.
- **Blue curve (test error)** falls at first and then *rises again*. Beyond the dashed line the model starts fitting noise, so it gets worse on unseen data.
- **Left of the dashed line:** both errors are high ⇒ underfitting. **Right of it:** a growing gap between training and test error ⇒ overfitting.
- **The dashed line** marks the minimum of the test error, the best complexity.

Rule of thumb: judge a model by the *test* curve, never by the training curve.

Common Mistake

Very high accuracy on training data does **not** mean the model is good. Always evaluate on data the model has never seen (Chapter 2).

	Overfitting	**Underfitting**
Model is	too complex	too simple
Training error	very low	high
Test error	high	high
Also called	high variance	high bias
Remedies	more training data; simpler model or fewer features; **regularisation**; remove noise/outliers; early stopping	more complex model; better or more features; reduce regularisation; train longer

Definition: Regularisation

Regularisation means deliberately *constraining* a model to make it simpler and so reduce overfitting, for example by adding a penalty on large parameter values to the loss (Chapter 4). The amount of regularisation is a hyperparameter.

1.7.3 Testing and Validating

The only reliable way to know how well a model generalises is to try it on new cases. So we split the data:

- **Training set** (e.g. 60–80%): used to fit the parameters.
- **Validation set** (e.g. 10–20%): used to compare models and tune *hyperparameters*.
- **Test set** (e.g. 10–20%): used *once*, at the very end, to estimate the **generalisation error**.

Common Mistake

If you tune hyperparameters on the *test* set, the test set is no longer "unseen": your reported error will be optimistic. Use the validation set (or cross-validation) for tuning and keep the test set locked away.

1.7.4 No Free Lunch Theorem

Wolpert (1996) proved that, averaged over *all possible problems*, no learning algorithm is better than any other. If we make *no assumption* about the data, there is no reason to prefer a linear

model over a neural network. In practice we must therefore (i) make reasonable **assumptions** about the data and (ii) *try several models* and compare them on a validation set.

Key Idea

The goal of ML is **generalisation**: performing well on *new* data, not on the data used for training.

Chapter Summary

- DL $\subset$ ML $\subset$ AI. ML learns *rules from data*; Mitchell's definition uses (T, P, E).
- Vocabulary: instance, feature, label, training/test set, model, parameters (learned) vs. hyperparameters (set by us).
- By supervision: supervised (labels), unsupervised (no labels), semi-supervised (few labels), reinforcement (rewards).
- Supervised tasks: classification (category) and regression (number). Unsupervised tasks: clustering, dimensionality reduction, anomaly detection.
- Batch learns offline from all data; online learns incrementally (learning rate matters). Instance-based memorises; model-based fits parameters.
- Challenges: insufficient data, non-representative data (sampling bias), poor quality, irrelevant features, overfitting, underfitting.
- Fix overfitting by more data, simpler models and regularisation; fix underfitting by richer models and better features.
- Split data into training, validation and test sets; never tune on the test set.

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define machine learning. **[2 marks]**
2. What is meant by (T, P, E) in Mitchell's definition? **[2 marks]**
3. Differentiate between a feature and a label. **[2 marks]**
4. Differentiate a parameter from a hyperparameter. **[2 marks]**
5. Name two applications each of classification and regression. **[2 marks]**
6. What is semi-supervised learning? Give one example. **[2 marks]**
7. Define overfitting. **[2 marks]**
8. What is the purpose of the test set? **[2 marks]**

Model Answer: Q2 and Q4

Q2. T is the *task* the program must perform, P is the *performance measure* used to judge it, and E is the *experience* (data) it learns from. A program learns if its P on T improves with E.

Q4. A *parameter* (e.g. θ_1 in linear regression) is learned automatically from the training data. A *hyperparameter* (e.g. k in k-NN or the learning rate) is set by the user before

training and tuned using a validation set.

B. Short Answer Questions (5 marks each)

1. Differentiate between AI, machine learning and deep learning with a suitable diagram. **[5 marks]**
2. Compare supervised and unsupervised learning with examples. **[5 marks]**
3. Explain batch and online learning. What is the role of the learning rate? **[5 marks]**
4. Differentiate between instance-based and model-based learning with examples. **[5 marks]**
5. Explain reinforcement learning with a diagram. Define agent, state, action, reward and policy. **[5 marks]**
6. Explain the bias in training data (sampling bias) with an example. **[5 marks]**

Model Answer: B2 (outline for 5 marks)

Supervised learning uses *labelled* data $(\boldsymbol{x}, y)$ and learns a mapping $f(\boldsymbol{x}) \approx y$; tasks are classification and regression (e.g. spam detection, house-price prediction); feedback is the correct answer. **Unsupervised learning** uses *unlabelled* data $\boldsymbol{x}$ and finds hidden structure; tasks are clustering, dimensionality reduction, anomaly detection (e.g. customer segmentation); there is no feedback.

Basis	**Supervised**	**Unsupervised**
Data	Labelled	Unlabelled
Goal	Predict output	Discover structure
Algorithms	k-NN, SVM, linear regression	K-Means, PCA, DBSCAN
Evaluation	Direct (compare with labels)	Indirect

C. Long Answer Questions (10 marks each)

1. Explain the different types of machine-learning systems based on the amount of supervision, with diagrams and examples. **[10 marks]**
2. Describe the main challenges of machine learning. **[10 marks]**
3. What are overfitting and underfitting? Explain with diagrams, state the causes and the remedies for each. **[10 marks]**
4. Using the data $(1, 15), (2, 26), (3, 29), (4, 44), (5, 46)$, fit a straight line $\hat{y} = \theta_0 + \theta_1 x$ by least squares and predict y for $x = 7$. **[10 marks]**

Model Answer: C4 (numerical)

$\bar{x} = 3,\ \bar{y} = 32.\ \sum(x-\bar{x})(y-\bar{y}) = 34+6+0+12+28 = 80;\ \sum(x-\bar{x})^2 = 4+1+0+1+4 = 10.$
$\theta_1 = 80/10 = 8,\ \ \theta_0 = 32 - 8(3) = 8.$ Model: $\hat{y} = 8 + 8x.$
For $x = 7$: $\hat{y} = 8 + 8(7) = \mathbf{64}$.

D. Multiple-Choice Questions

1. Which of the following is *not* a type of ML system by supervision?
   (a) Supervised (b) Unsupervised (c) Reinforcement (d) **Compiled**

2. Grouping customers by buying behaviour without predefined labels is:
   (a) classification (b) regression (c) clustering (d) reinforcement
3. A model with very low training error but high test error is:
   (a) underfitting (b) overfitting (c) well fitted (d) unbiased
4. In Mitchell's definition, accuracy of a spam filter is an example of:
   (a) task (b) experience (c) performance measure (d) model
5. k-Nearest Neighbours is an example of:
   (a) model-based learning (b) instance-based learning (c) online learning (d) RL
6. The discount factor γ is used in:
   (a) clustering (b) regression (c) reinforcement learning (d) PCA
7. Hyperparameters are tuned using the:
   (a) training set (b) validation set (c) test set (d) none

Answers: 1-d, 2-c, 3-b, 4-c, 5-b, 6-c, 7-b.

E. Practice Problems

1. Label each as classification, regression or clustering: (a) predicting tomorrow's temperature, (b) deciding whether a tumour is benign, (c) grouping news articles by topic.
2. Write the (T, P, E) description for a handwriting-recognition program.
3. A robot has $\gamma = 0.9$ and receives rewards 1, 0, 5 at the next three steps (and none afterwards). Compute G_t. *(Answer:* $1 + 0.9 \cdot 0 + 0.81 \cdot 5 = 5.05$*.)*
4. **Coding:** extend the `LinearRegression` program to print predictions for sizes 6, 7 and 8. Plot the data and the fitted line with `matplotlib`.

Quick Revision

Topic	One-line recall
Samuel (1959)	Learning without being explicitly programmed.
Mitchell (1997)	Performance P at task T improves with experience E.
Supervised	Labelled data; classification (category) / regression (number).
Unsupervised	No labels; clustering, PCA, anomaly detection.
Semi-supervised	Few labels + many unlabelled (self-training).
Reinforcement	Agent–environment loop; maximise $G_t = \sum \gamma^k r_{t+k+1}$.
Batch / Online	All data once / incremental; learning rate.
Instance / Model-based	Memorise and compare / fit parameters.
Overfitting	Low train error, high test error $\rightarrow$ more data, regularise, simplify.
Underfitting	High train and test error $\rightarrow$ richer model, better features.
Data splits	Train: fit. Validation: tune. Test: final estimate.
No Free Lunch	No best algorithm for all problems; make assumptions and compare.

Chapter 2

Statistical Learning

Learning Objectives

After studying this chapter you will be able to:

- describe the statistical model $Y = f(X) + \varepsilon$ and explain *prediction* versus *inference*;
- state the statistical view of **supervised** and **unsupervised** learning;
- define and compute common **loss functions** (squared, absolute, 0–1);
- distinguish **training loss**, **test loss** and **risk**, and explain why training loss is optimistic;
- derive and interpret the **bias–variance decomposition** and describe the main **trade-offs** of statistical learning;
- estimate risk using the **hold-out**, **k-fold** and **leave-one-out** methods and compute classification statistics (accuracy, precision, recall, F1);
- explain the **sampling distribution of an estimator**, its bias, variance and standard error, and use the Central Limit Theorem;
- answer 2-, 5- and 10-mark university questions on Unit I.

> *All models are wrong, but some are useful.*
>
> — George Box

2.1 Introduction to Statistical Learning

Chapter 1 told us *what* machine learning is. **Statistical learning** is the mathematical framework that tells us *why it works*, *how well* it works, and *how to measure that*. It treats data as random samples drawn from some unknown real-world process and asks: how can we learn that process from a finite sample?

2.1.1 The Statistical Model

Suppose we observe p input variables $X = (X_1, \ldots, X_p)$ (the *features*) and one output variable Y (the *response*). We assume that they are related by

$$Y = f(X) + \varepsilon. \tag{2.1}$$

- f is an **unknown fixed function**: the systematic, predictable part of the relationship.
- ε is a **random error** (noise) with mean zero, $\mathbb{E}[\varepsilon] = 0$, independent of X and with variance σ^2. It stands for everything we did not measure or that is genuinely random.

Statistical learning is therefore *the set of methods for estimating* f from data. Our estimate is written $\hat{f}$.

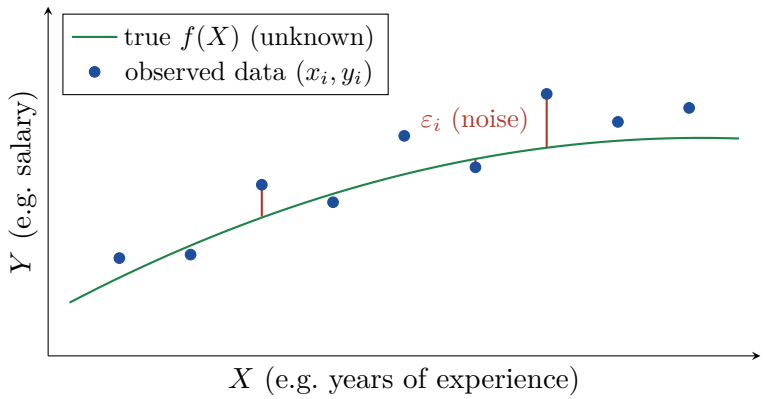


Figure 2.1. Data are generated as $Y = f(X) + \varepsilon$. The curve is the true but unknown f; dots are noisy observations.

Reading the Figure

The **green curve** is the true relationship $f(X)$, which in real life we never see. The **blue dots** are what we actually collect: each dot lies slightly above or below the curve because of random noise. The **short red vertical segments** are examples of the noise term $\varepsilon_i = y_i - f(x_i)$. Our job is to look only at the dots and recover a curve $\hat{f}$ as close to the green one as possible. Because of the noise, we can *never* recover f perfectly.

2.1.2 Why Estimate f? Prediction and Inference

Prediction. We only care about getting accurate predictions $\hat{Y} = \hat{f}(X)$ for new X. $\hat{f}$ can be a "black box". *Example:* forecasting tomorrow's sales.

Inference. We want to *understand* how Y changes with each X_j: which features matter, whether the effect is positive or negative, and how strong it is. $\hat{f}$ must be interpretable. *Example:* how much does one extra year of education raise salary?

Reducible and irreducible error. For a fixed $\hat{f}$ and X,

$$\mathbb{E}\big[(Y - \hat{Y})^2\big] = \underbrace{\big[f(X) - \hat{f}(X)\big]^2}_{\text{reducible}} + \underbrace{\operatorname{Var}(\varepsilon)}_{\text{irreducible}=\sigma^2}. \tag{2.2}$$

Better methods shrink the reducible part by making $\hat{f}$ closer to f, but *no* method can remove the irreducible part σ^2. This is the lower limit on any model's error.

2.1.3 Parametric and Non-parametric Methods

	Parametric	**Non-parametric**
Idea	Assume a form for f, e.g. $f(X) = \theta_0 + \theta_1 X$, then estimate a few parameters	Make no strong assumption about f; let the data decide its shape
Advantage	Simple, needs little data, easy to interpret	Flexible; can fit any shape
Disadvantage	Wrong assumption $\Rightarrow$ poor fit	Needs a lot of data; risk of over-fitting
Examples	Linear / logistic regression, Naive Bayes	k-NN, decision trees

2.2 Supervised and Unsupervised Learning: The Statistical View

2.2.1 Supervised Learning

We observe n pairs $(x_1, y_1), \ldots, (x_n, y_n)$ drawn *independently* from an unknown joint distribution $P(X, Y)$ (this is the **i.i.d. assumption**: independent and identically distributed). We use them to learn a function that predicts Y from X.

Key Idea

What is the best possible predictor?

- For **regression** with squared error, the best predictor is the *conditional mean* $f^*(x) = \mathbb{E}[Y \mid X = x]$ (the **regression function**).
- For **classification**, the best predictor is the **Bayes classifier**: assign x to the class with the highest posterior probability, $\hat{y} = \arg\max_k P(Y = k \mid X = x)$.

Every supervised method is an attempt to approximate one of these using only a finite sample.

The error of the Bayes classifier is the **Bayes error rate**. It is the irreducible error for classification: even the perfect classifier is wrong sometimes when classes overlap.

2.2.2 Unsupervised Learning

We observe only $x_1, \ldots, x_n$ and no y. Statistically, the goal is to learn the structure of the distribution $P(X)$ itself:

- **density estimation:** estimate $p(x)$;
- **clustering:** find groups (modes) in $p(x)$;
- **dimensionality reduction:** find a low-dimensional description of X that retains most of its variation.

Since there is no "correct answer" to compare against, evaluating unsupervised methods is harder and often relies on domain judgement.

	Supervised	**Unsupervised**
Data	(x_i, y_i) pairs	x_i only
Learns	$P(Y \mid X)$ or f	$P(X)$ / structure
Goal	Predict Y for new X	Discover patterns
Loss	Prediction error vs. true y	No direct loss (e.g. within-cluster variance)

2.3 Loss Functions, Training Loss and Test Loss

2.3.1 Loss Functions

To say how good $\hat{f}$ is, we need a number measuring the cost of a wrong prediction.

Definition: Loss function

A **loss function** $L(y, \hat{y}) \geq 0$ measures how bad the prediction $\hat{y}$ is when the true value is y. Lower is better, and $L(y, y) = 0$.

Loss	**Formula**	**Used for / property**
Squared error (L2)	$(y - \hat{y})^2$	Regression; punishes large errors heavily; smooth
Absolute error (L1)	$\lvert y - \hat{y} \rvert$	Regression; robust to outliers
0–1 loss	$\mathbb{1}[y \neq \hat{y}]$	Classification; counts mistakes
Cross-entropy (log loss)	$-\sum_k \mathbb{1}[y = k] \ln \hat{p}_k$	Probabilistic classifiers (Chapter 4, 9)

Example: Computing losses

True values $y = (3, 5, 7, 9)$; predictions $\hat{y} = (2, 6, 7, 11)$.

i	1	2	3	4	**Sum**
$y_i - \hat{y}_i$	1	-1	0	-2	
$\lvert y_i - \hat{y}_i \rvert$	1	1	0	2	4
$(y_i - \hat{y}_i)^2$	1	1	0	4	6

$$\text{MAE} = \tfrac{4}{4} = 1, \qquad \text{MSE} = \tfrac{6}{4} = 1.5, \qquad \text{RMSE} = \sqrt{1.5} \approx 1.22.$$

Observation: the single error of 2 contributes $2/4 = 50\%$ of the absolute error but $4/6 \approx 67\%$ of the squared error. Squared loss emphasises large errors.

2.3.2 Risk, Training Loss and Test Loss

Definition: Risk (expected loss, generalisation error)

The **risk** of a function f is its *expected* loss on a new observation drawn from the true distribution:

$$\mathcal{R}(f) = \mathbb{E}_{(X,Y)}[L(Y, f(X))].$$

The risk is exactly what we want to minimise, but it needs the unknown $P(X, Y)$, so we can never compute it. We estimate it from samples.

Definition: Training loss (empirical risk)

For the training sample $(x_i, y_i)_{i=1}^n$,

$$\hat{\mathcal{R}}_{\text{train}}(f) = \frac{1}{n}\sum_{i=1}^{n} L(y_i, f(x_i)).$$

Empirical risk minimisation (ERM): choose $\hat{f} = \arg\min_{f \in \mathcal{F}} \hat{\mathcal{R}}_{\text{train}}(f)$ over a chosen family $\mathcal{F}$ of models.

Definition: Test loss

For a *separate* sample $(x'_j, y'_j)_{j=1}^m$ that was **not** used for training,

$$\hat{\mathcal{R}}_{\text{test}}(\hat{f}) = \frac{1}{m}\sum_{j=1}^{m} L(y'_j, \hat{f}(x'_j)).$$

It is an *honest* (unbiased) estimate of the risk $\mathcal{R}(\hat{f})$.

Common Mistake

Training loss is an optimistic estimate of risk. The model was *chosen to minimise it* on exactly those points, so it fits them (and their noise) better than it will fit new data. Test loss $\geq$ training loss on average. Never report training loss as the performance of a model.

Try It Yourself

See the gap between training and test loss for polynomial fits of degree 1, 3 and 9 to noisy data from $y = \sin(2\pi x)$:

```
import numpy as np
rng = np.random.default_rng(0)
x = np.sort(rng.uniform(0, 1, 30))
y = np.sin(2*np.pi*x) + rng.normal(0, 0.25, 30)    # f + noise
idx = rng.permutation(30)
tr, te = idx[:20], idx[20:]                          # 20 train / 10 test

def mse(coef, i):
    return np.mean((np.polyval(coef, x[i]) - y[i])**2)

for d in [1, 3, 9]:
    coef = np.polyfit(x[tr], y[tr], d)               # ERM for degree d
    print(d, round(mse(coef, tr), 3), round(mse(coef, te), 3))
```

Output (seed 0):

Degree	Train MSE	Test MSE	Verdict
1	0.222	0.255	underfits (both high)
3	0.041	0.050	good fit
9	0.026	0.102	overfits (test $\approx 4\times$ train)

Notice that the degree-9 model has the *lowest* training loss but a test loss twice that of degree 3. Training loss alone would have picked the wrong model.

2.3.3 Splitting the Data

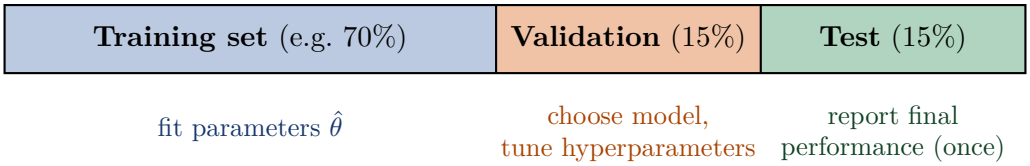


Figure 2.2. Splitting a dataset into training, validation and test sets.

Reading the Figure

The long bar is the *whole* dataset, cut into three **non-overlapping** parts. The label under each part says its single job. The **blue** part is the only one the learning algorithm is allowed to fit. The **orange** part is used to *compare* models and choose hyperparameters. The **green** part is kept locked away until the very end and used *once* to report the final honest performance. Mixing these jobs (for example, tuning on the test part) makes the final number over-optimistic.

2.4 Trade-offs in Statistical Learning

2.4.1 The Bias–Variance Trade-off

Take a *fixed* test point x_0. Imagine collecting many different training sets, each of size n, and fitting $\hat{f}$ on each one. The predictions $\hat{f}(x_0)$ vary from set to set. The expected squared prediction error at x_0 then splits into three parts:

$$\mathbb{E}\Big[(y_0 - \hat{f}(x_0))^2\Big] = \underbrace{\Big(\mathbb{E}[\hat{f}(x_0)] - f(x_0)\Big)^2}_{\mathrm{Bias}(\hat{f}(x_0))^2} + \underbrace{\mathbb{E}\Big[(\hat{f}(x_0) - \mathbb{E}[\hat{f}(x_0)])^2\Big]}_{\mathrm{Var}(\hat{f}(x_0))} + \underbrace{\sigma^2}_{\text{irreducible}}. \tag{2.3}$$

Definition: Bias and variance of a method

Bias is the error caused by approximating a complicated real problem by a simpler model: how far the *average* prediction is from the truth. **Variance** is how much the prediction would *change* if we trained on a different sample.

Proof sketch of Equation (2.3). Write $y_0 = f + \varepsilon$ and let $\bar{f} = \mathbb{E}[\hat{f}]$. Add and subtract $\bar{f}$:

$$y_0 - \hat{f} = (f - \bar{f}) + (\bar{f} - \hat{f}) + \varepsilon.$$

Square and take expectations. The cross-terms vanish because $\mathbb{E}[\bar{f} - \hat{f}] = 0$ and $\mathbb{E}[\varepsilon] = 0$ with ε independent of $\hat{f}$. What remains is $(f - \bar{f})^2 + \mathbb{E}[(\hat{f} - \bar{f})^2] + \mathbb{E}[\varepsilon^2]$, i.e. $\mathrm{Bias}^2 + \mathrm{Var} + \sigma^2$. ∎

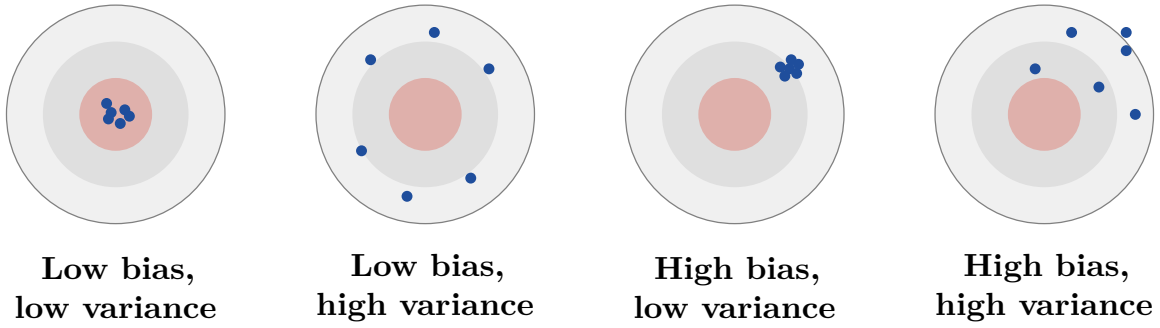


Figure 2.3. Bias and variance explained with a dartboard. Red centre = true value $f(x_0)$; blue dots = predictions from models trained on different samples.

Reading the Figure

Think of each **blue dot** as the prediction made by a model trained on a different random sample, and the **red bull's-eye** as the true value.

- *Low bias, low variance:* dots are tightly clustered *on* the centre. This is the ideal.
- *Low bias, high variance:* dots are centred correctly on average but scattered. Predictions depend heavily on the particular sample (**overfitting**).
- *High bias, low variance:* dots are tightly clustered but *away* from the centre. The model is consistently wrong in the same way (**underfitting**).
- *High bias, high variance:* dots are scattered and off-centre: the worst case.

Bias is the distance from the *centre of the dots* to the bull's-eye; *variance* is the *spread* of the dots.

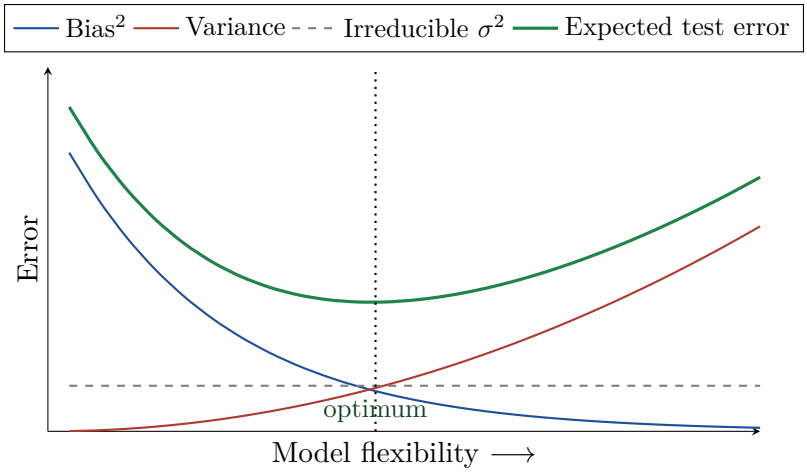


Figure 2.4. Bias^2 falls and variance rises as flexibility increases; expected test error is their sum plus noise.

Reading the Figure

The horizontal axis is how flexible the model is (e.g. polynomial degree, or $1/k$ in k-NN). Read the four curves:

- **Blue (bias^2)** starts high and falls: a flexible model can follow the true f more closely.
- **Red (variance)** starts near zero and rises: a flexible model chases the noise in each

training sample.

- **Grey dashed (irreducible error)** is a flat floor that no model can beat.
- **Green (expected test error)** is the *sum* of the three. It is U-shaped: its lowest point (dotted line) is the best flexibility.

Left of the optimum, bias dominates (underfitting); right of it, variance dominates (overfitting). Compare this with Figure 1.8 in Chapter 1.

Example: Putting numbers on the decomposition

At a test point, a model has $\text{Bias}^2 = 4$, $\text{Var} = 3$ and noise $\sigma^2 = 2$.

$$\text{Expected test MSE} = 4 + 3 + 2 = 9.$$

If we switch to a more flexible model with $\text{Bias}^2 = 1$ and $\text{Var} = 5$, the error becomes $1 + 5 + 2 = 8$: lower, even though variance rose, because the bias fell more. If flexibility is pushed further to $\text{Bias}^2 = 0.5$, $\text{Var} = 9$, error is $0.5 + 9 + 2 = 11.5$: worse. This is the trade-off.

2.4.2 Other Trade-offs

1. **Flexibility vs. interpretability.** Simple models (linear regression) are easy to explain but may be too rigid; flexible models (boosting, deep networks) can fit complex patterns but act as black boxes.
2. **Prediction accuracy vs. inference.** If you need to *explain* effects, prefer simple models; if you only need accurate predictions, flexible models are fine.
3. **Model complexity vs. amount of data.** Complex models need much more data. With little data, a simple model usually generalises better.
4. **Training cost vs. accuracy.** More flexible models and more data take longer to train and need more memory.

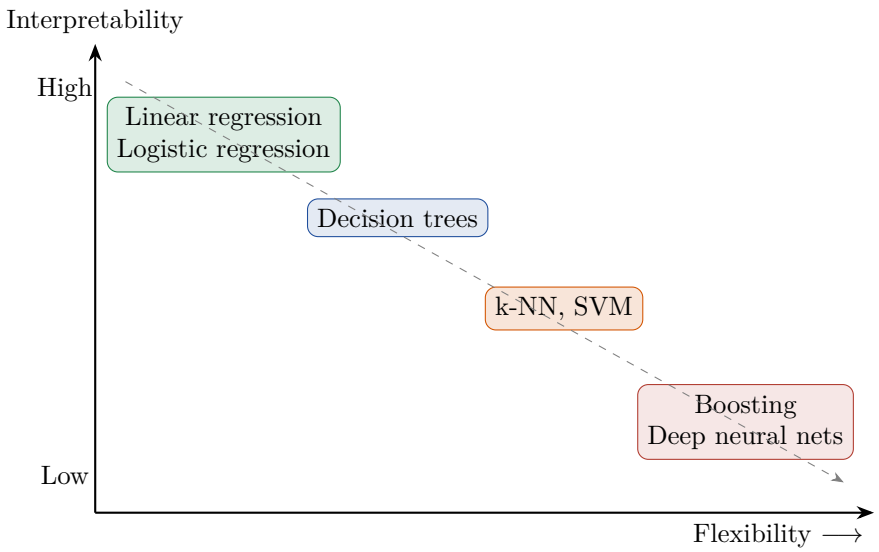


Figure 2.5. Typical position of methods on the flexibility–interpretability trade-off (qualitative).

Reading the Figure

Each box is a family of methods placed by two properties. Moving **right**, models become more flexible (can fit more complicated shapes). Moving **down**, they become harder to interpret. The dashed arrow shows the general trend: *what you gain in flexibility you usually pay for in interpretability*. The positions are rough guides, not exact measurements; these methods are covered in Chapters 3–9.

Exam Tip

The bias–variance trade-off is the single most-asked topic of Unit I. Be ready to (i) state Equation (2.3), (ii) sketch Figure 2.4 with the four curves, and (iii) link high bias to underfitting and high variance to overfitting.

2.5 Estimating Risk Statistics

We cannot compute the true risk $\mathcal{R}(\hat{f})$, so we *estimate* it. A good estimate should be nearly unbiased and have low variance.

2.5.1 Hold-out (Validation-set) Estimate

Split once into training and test parts and compute $\hat{\mathcal{R}}_{\text{test}}$. It is simple and fast, but (i) the number depends on *which* points fell in the test set, and (ii) the model is trained on less data, so it may look slightly worse than it is.

Uncertainty of the estimate. If the m test losses $\ell_j = L(y'_j, \hat{f}(x'_j))$ have sample mean $\bar{\ell} = \hat{\mathcal{R}}_{\text{test}}$ and sample standard deviation s, then

$$\text{SE}(\bar{\ell}) = \frac{s}{\sqrt{m}}, \qquad \text{approximate 95\% CI for the risk:} \quad \bar{\ell} \pm 1.96\ \text{SE}(\bar{\ell}).$$

Example: Confidence interval for test MSE

On $m = 100$ test points the squared errors have mean 4.1 and standard deviation $s = 3$. Then $\text{SE} = 3/\sqrt{100} = 0.3$ and the 95% interval is $4.1 \pm 1.96(0.3) = 4.1 \pm 0.59$, i.e. [3.51, 4.69]. So we may report "test MSE ≈ 4.1, likely between 3.5 and 4.7". Quadrupling the test set to $m = 400$ would halve the standard error to 0.15.

2.5.2 k-Fold Cross-Validation

When data are scarce, we want to use *all* of it for both training and evaluation.

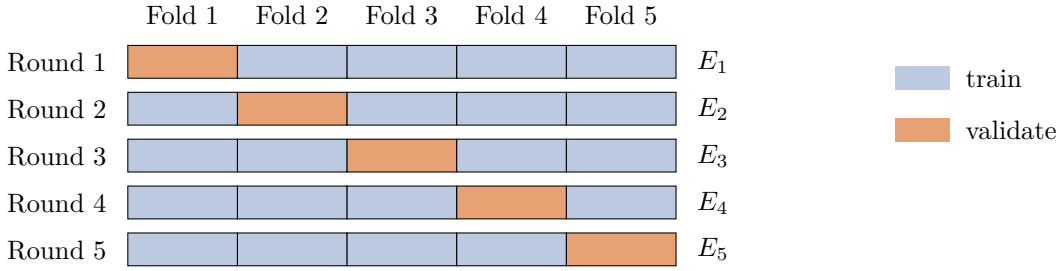


Figure 2.6. 5-fold cross-validation. Each round uses a different fold (orange) for validation and the other four (blue) for training.

Reading the Figure

The dataset is cut into $k = 5$ equal **folds** (the columns). Each **row** is one training round. In Round 1 the model is trained on Folds 2–5 (blue) and its error E_1 is measured on Fold 1 (orange). In Round 2 the orange fold moves to Fold 2, and so on. After five rounds *every point has been used for validation exactly once*, and the final estimate is the average $E_1, \ldots, E_5$ of the five errors.

Definition: k-fold cross-validation estimate

Split the data into k folds. For $i = 1, \ldots, k$ train on all folds except the i-th and compute the error E_i on the i-th. Then

$$\mathrm{CV}_{(k)} = \frac{1}{k} \sum_{i=1}^{k} E_i.$$

Example: 5-fold CV with numbers

Validation errors: $E = (4.2,\ 3.8,\ 4.6,\ 4.0,\ 3.9)$.

$$\mathrm{CV}_{(5)} = \frac{4.2 + 3.8 + 4.6 + 4.0 + 3.9}{5} = \frac{20.5}{5} = 4.1.$$

Deviations from 4.1: $0.1, -0.3, 0.5, -0.1, -0.2$; squares sum to 0.40; $s^2 = 0.40/4 = 0.10$, $s = 0.316$; SE $= s/\sqrt{5} = 0.141$. So the estimated risk is 4.1 ± 0.14.

Choosing k. **Leave-one-out CV (LOOCV)** is the extreme $k = n$: nearly unbiased but expensive and with high variance. In practice $k = 5$ or $k = 10$ gives a good bias–variance balance. For classification use *stratified* folds, which keep the class proportions the same in each fold.

	Hold-out	**k-fold CV**	**LOOCV**
Number of fits	1	k	n
Computation	Cheapest	Moderate	Most expensive
Bias of estimate	Higher (less training data)	Low–moderate	Lowest
Variance of estimate	High (one split)	Moderate	High
Typical use	Large datasets	Default choice	Very small datasets

2.5.3 Risk Statistics for Classification

For classification with 0–1 loss, the risk is the **error rate**. We summarise the test predictions in a **confusion matrix**.

		Predicted	
		Positive	Negative
Actual	Positive	TP (true positive)	FN (false negative)
	Negative	FP (false positive)	TN (true negative)

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN},$$

$$\text{Specificity} = \frac{TN}{TN + FP}, \qquad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Precision answers "of the items I flagged, how many were truly positive?"; *recall* answers "of all true positives, how many did I find?"; F_1 is their harmonic mean.

Example: Spam filter on 100 test e-mails

Actual spam $= 50$, actual non-spam $= 50$. The filter finds $TP = 40$ spam mails, misses $FN = 10$, wrongly flags $FP = 5$ good mails and correctly passes $TN = 45$.

$$\text{Accuracy} = \tfrac{40+45}{100} = 0.85, \qquad \text{Precision} = \tfrac{40}{45} \approx 0.889,$$
$$\text{Recall} = \tfrac{40}{50} = 0.80, \qquad \text{Specificity} = \tfrac{45}{50} = 0.90,$$
$$F_1 = \tfrac{2(0.889)(0.80)}{0.889+0.80} \approx 0.842.$$

Common Mistake

Accuracy can mislead on imbalanced data. If only 1% of transactions are fraud, a program that always says "not fraud" is 99% accurate yet has recall 0 and is useless. Use precision, recall and F_1 instead.

2.6 Sampling Distribution of an Estimator

Everything we compute from data (a mean, a test error, a regression coefficient) is a *statistic* calculated from a random sample. Draw a different sample and you get a different number. This section studies that randomness.

Definition: Estimator and estimate

An **estimator** $\hat{\theta} = g(X_1, \ldots, X_n)$ is a *rule* (a function of the sample) for guessing an unknown population quantity θ. An **estimate** is the *number* that the rule gives for one particular sample. Since the sample is random, the estimator $\hat{\theta}$ is itself a **random variable**.

Definition: Sampling distribution

The **sampling distribution** of an estimator is its probability distribution over all possible samples of size n from the population. Its *standard deviation* is called the **standard error**, $\text{SE}(\hat{\theta}) = \sqrt{\text{Var}(\hat{\theta})}$.

2.6.1 Properties of Good Estimators

Property	Definition	Meaning
Bias	$\mathrm{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$	Average error; **unbiased** if $= 0$
Variance	$\mathrm{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}\hat{\theta})^2]$	Spread across samples
Standard error	$\mathrm{SE}(\hat{\theta}) = \sqrt{\mathrm{Var}(\hat{\theta})}$	Typical size of the estimation error
MSE	$\mathbb{E}[(\hat{\theta} - \theta)^2] = \mathrm{Var}(\hat{\theta}) + \mathrm{Bias}(\hat{\theta})^2$	Overall accuracy (same decomposition as before!)
Consistency	$\hat{\theta} \to \theta$ as $n \to \infty$	More data gives a better estimate

2.6.2 The Sample Mean

Let $X_1, \ldots, X_n$ be i.i.d. with population mean μ and variance σ^2, and let $\bar{X} = \frac{1}{n}\sum X_i$. Then

$$\mathbb{E}[\bar{X}] = \mu \quad \text{(unbiased)}, \qquad \mathrm{Var}(\bar{X}) = \frac{\sigma^2}{n}, \qquad \mathrm{SE}(\bar{X}) = \frac{\sigma}{\sqrt{n}}.$$

Proof. $\mathbb{E}[\bar{X}] = \frac{1}{n}\sum \mathbb{E}[X_i] = \frac{1}{n}(n\mu) = \mu$, and, by independence, $\mathrm{Var}(\bar{X}) = \frac{1}{n^2}\sum \mathrm{Var}(X_i) = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n}$. ■

Key Idea

The standard error shrinks like $1/\sqrt{n}$. To make an estimate *twice* as precise you need *four times* as much data.

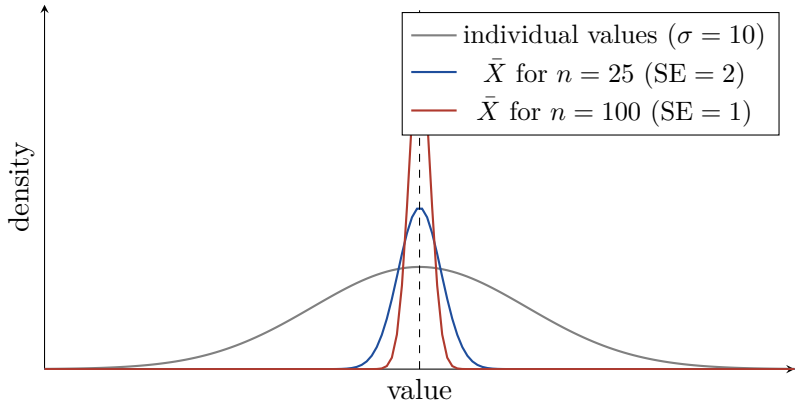


Figure 2.7. Sampling distribution of the sample mean for population mean $\mu = 50$ and $\sigma = 10$.

Reading the Figure

Three bell curves are drawn, all centred on the true mean $\mu = 50$ (dashed line).

- **Grey (widest):** the spread of *individual* observations, $\sigma = 10$.
- **Blue:** the distribution of the *average of 25 observations*. It is still centred at 50 (*unbiased*) but is 5× narrower, $\mathrm{SE} = 10/\sqrt{25} = 2$.
- **Red (narrowest and tallest):** the average of 100 observations, $\mathrm{SE} = 1$.

The *height* of a curve at a value shows how likely it is that the estimate lands there. The bigger the sample, the more tightly the estimates cluster around the truth. Averages are much more reliable than single observations.

Example: Using the standard error

A population has $\mu = 50$ and $\sigma = 10$. For samples of size $n = 25$, what is $P(\bar{X} > 53)$? $\text{SE} = 10/\sqrt{25} = 2$, so $z = \dfrac{53 - 50}{2} = 1.5$ and $P(\bar{X} > 53) = P(Z > 1.5) \approx 0.0668$, about 6.7%. With $n = 100$, $\text{SE} = 1$, $z = 3$, and the probability drops to about 0.0013.

2.6.3 Central Limit Theorem

Definition: Central Limit Theorem (CLT)

Let $X_1, \ldots, X_n$ be i.i.d. with mean μ and finite variance σ^2. For large n the sampling distribution of $\bar{X}$ is approximately normal:

$$\bar{X} \;\approx\; \mathcal{N}\!\left(\mu,\; \frac{\sigma^2}{n}\right), \qquad \text{equivalently} \qquad \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \;\approx\; \mathcal{N}(0, 1),$$

whatever the shape of the original population.

The CLT is the reason the normal curve appears everywhere: test errors, accuracies and averages of many small effects are sums of many small random pieces. It justifies the confidence intervals used in the previous section (a rule of thumb: $n \geq 30$ is usually enough).

95% confidence interval for a mean. $\bar{x} \pm 1.96 \dfrac{s}{\sqrt{n}}$.

Example: Confidence interval

A sample of $n = 100$ gives $\bar{x} = 52$ and $s = 10$. $\text{SE} = 10/10 = 1$, so the 95% interval is 52 ± 1.96, i.e. $[50.04, 53.96]$. If we repeated the experiment many times, about 95% of intervals built this way would contain the true μ.

2.6.4 A Biased Estimator: the Variance

Not all natural estimators are unbiased. Consider estimating σ^2:

$$S_n^2 = \frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^2, \qquad S_{n-1}^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2.$$

It can be shown that $\mathbb{E}[S_n^2] = \dfrac{n-1}{n}\sigma^2 < \sigma^2$, so S_n^2 is **biased** (it underestimates), while S_{n-1}^2 is unbiased. The reason is that deviations are measured from $\bar{X}$, which is itself fitted to the same data and so sits *closer* to the sample than the true μ does. This is the same "fits its own data too well" effect that makes training loss optimistic.

Example: Size of the bias

With $n = 5$ and $\sigma^2 = 10$: $\mathbb{E}[S_n^2] = \frac{4}{5}(10) = 8$ (bias $= -2$), whereas $\mathbb{E}[S_{n-1}^2] = 10$.

Try It Yourself

Verify the standard-error formula and the bias of S_n^2 by simulation:

```python
import numpy as np
rng = np.random.default_rng(1)

for n in [5, 25, 100]:
    means = [rng.normal(50, 10, n).mean() for _ in range(10000)]
    print(n, round(np.mean(means), 2), round(np.std(means), 2),
          round(10/np.sqrt(n), 2))      # theory: SE = sigma/sqrt(n)

v0 = [np.var(rng.normal(0, np.sqrt(10), 5), ddof=0) for _ in range(20000)]
v1 = [np.var(rng.normal(0, np.sqrt(10), 5), ddof=1) for _ in range(20000)]
print(round(np.mean(v0), 2), round(np.mean(v1), 2))
```

Typical output:

n	mean of $\bar{X}$	std of $\bar{X}$	theory $10/\sqrt{n}$
5	49.92	4.39	4.47
25	49.98	2.00	2.00
100	50.00	1.01	1.00

The means stay near 50 (unbiased) and the spread follows $\sigma/\sqrt{n}$. The two variance estimates average about 8.0 (ddof = 0, biased) and 9.9 (ddof = 1, unbiased, close to the true 10).

Key Idea

Estimators (including a trained model's predictions and its test error) are random. To trust a number, we need to know **how much it would vary from sample to sample**: that is its **standard error**.

Chapter Summary

- Statistical model: $Y = f(X) + \varepsilon$; we estimate f for *prediction* or *inference*. Error = reducible + irreducible (σ^2).
- Supervised learning estimates $P(Y \mid X)$ (best predictors: conditional mean, Bayes classifier). Unsupervised learning studies $P(X)$.
- Loss measures a single mistake (squared, absolute, 0–1, cross-entropy); **risk** is expected loss; **training loss** (empirical risk) is optimistic; **test loss** is an honest estimate.
- $\mathbb{E}[(y_0 - \hat{f}(x_0))^2] = \text{Bias}^2 + \text{Var} + \sigma^2$. More flexibility lowers bias but raises variance; the best model balances them.
- Other trade-offs: flexibility vs. interpretability, prediction vs. inference, complexity vs. data size.
- Estimate risk by hold-out, k-fold CV ($k = 5$ or 10) or LOOCV; for classification use the confusion matrix, precision, recall and F_1.
- An estimator is a random variable; its spread is the **standard error**. For the sample

mean SE $= \sigma/\sqrt{n}$, and the **CLT** makes $\bar{X}$ approximately normal.

- S_n^2 (divide by n) is biased; S_{n-1}^2 is unbiased.

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Write the statistical model $Y = f(X) + \varepsilon$ and name each term. **[2 marks]**
2. Differentiate between prediction and inference. **[2 marks]**
3. What is irreducible error? **[2 marks]**
4. Define a loss function. Give two examples. **[2 marks]**
5. Why is training loss an optimistic estimate of the risk? **[2 marks]**
6. Define bias and variance of a learning method. **[2 marks]**
7. What is k-fold cross-validation? **[2 marks]**
8. Define standard error of an estimator. **[2 marks]**
9. State the Central Limit Theorem. **[2 marks]**
10. Define precision and recall. **[2 marks]**

Model Answer: Q5 and Q8

Q5. The model parameters are chosen to *minimise the loss on the training data itself*, so the model partly memorises its noise; on fresh data it performs worse. Hence training loss $\leq$ expected test loss.
Q8. The standard error of an estimator $\hat{\theta}$ is the standard deviation of its sampling distribution, $\mathrm{SE}(\hat{\theta}) = \sqrt{\mathrm{Var}(\hat{\theta})}$. For the sample mean it equals $\sigma/\sqrt{n}$.

B. Short Answer Questions (5 marks each)

1. Distinguish parametric and non-parametric methods with examples. **[5 marks]**
2. Explain training loss, test loss and risk. Why do we need a separate test set? **[5 marks]**
3. Explain the bias–variance trade-off with a diagram. **[5 marks]**
4. Explain k-fold cross-validation with a diagram. Compare it with the hold-out method. **[5 marks]**
5. Explain the confusion matrix and define accuracy, precision, recall and F_1. **[5 marks]**
6. What is the sampling distribution of an estimator? Show that the sample mean is unbiased with $\mathrm{Var}(\bar{X}) = \sigma^2/n$. **[5 marks]**
7. Why is accuracy a poor measure for imbalanced data? Illustrate with an example. **[5 marks]**

Model Answer: B6 (outline for 5 marks)

An *estimator* $\hat{\theta} = g(X_1, \ldots, X_n)$ is a function of a random sample, hence a random variable; its probability distribution over repeated samples of size n is its *sampling distribution*.
For i.i.d. X_i with mean μ and variance σ^2: $\mathbb{E}[\bar{X}] = \frac{1}{n}\sum \mathbb{E}[X_i] = \mu$, so $\bar{X}$ is unbiased. By

independence, $\mathrm{Var}(\bar{X}) = \frac{1}{n^2}\sum \mathrm{Var}(X_i) = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n}$. Hence $\mathrm{SE}(\bar{X}) = \sigma/\sqrt{n}$; precision improves as n grows. By the CLT, $\bar{X} \approx \mathcal{N}(\mu, \sigma^2/n)$ for large n.

C. Long Answer Questions (10 marks each)

1. Derive the bias–variance decomposition of the expected squared prediction error. Explain its implications for model selection and sketch the curves. **[10 marks]**
2. Describe the different methods of estimating the risk of a model (hold-out, k-fold, LOOCV) with their advantages and disadvantages. **[10 marks]**
3. Explain the sampling distribution of the sample mean. State the CLT and explain, with an example, how it is used to construct a confidence interval. **[10 marks]**
4. Compare supervised and unsupervised learning from a statistical point of view, and discuss the trade-offs in statistical learning. **[10 marks]**
5. A classifier is tested on 200 samples: $TP = 70$, $FN = 30$, $FP = 20$, $TN = 80$. Compute accuracy, precision, recall, specificity and F_1. **[10 marks]**

Model Answer: C5 (numerical)

$n = 200$. Accuracy $= \frac{70+80}{200} = 0.75$. Precision $= \frac{70}{70+20} = 0.778$. Recall $= \frac{70}{70+30} = 0.70$. Specificity $= \frac{80}{80+20} = 0.80$.
$F_1 = \frac{2(0.778)(0.70)}{0.778+0.70} = \frac{1.089}{1.478} \approx \mathbf{0.737}$.

D. Numerical Problems

1. True $y = (2, 4, 6, 8)$, predicted $\hat{y} = (3, 3, 6, 10)$. Compute MAE, MSE and RMSE. *(Answer: $MAE = 1$, $MSE = 1.5$, $RMSE = 1.22$.)*
2. A model has $\mathrm{Bias}^2 = 2.5$, $\mathrm{Var} = 1.5$, $\sigma^2 = 1$. Find the expected test MSE. After switching to a simpler model, $\mathrm{Bias}^2 = 4$, $\mathrm{Var} = 0.5$: which is better? *(Answer: 5; the new one gives 5.5, so the first is better.)*
3. 4-fold CV errors are 6, 5, 7, 6. Find the CV estimate and its standard error. *(Answer: 6; $s = 0.816$, $\mathrm{SE} = 0.408$.)*
4. A population has $\mu = 100$ and $\sigma = 20$. For $n = 64$, find $\mathrm{SE}(\bar{X})$ and $P(\bar{X} < 97)$. *(Answer: $\mathrm{SE} = 2.5$; $z = -1.2$; $P \approx 0.115$.)*
5. Test squared errors over 400 samples have mean 5 and $s = 4$. Give a 95% CI for the risk. *(Answer: $\mathrm{SE} = 0.2$; $5 \pm 0.392 = [4.61, 5.39]$.)*

E. Multiple-Choice Questions

1. The irreducible error in $Y = f(X) + \varepsilon$ is:
   (a) Bias^2 (b) $\mathrm{Var}(\hat{f})$ (c) $\mathrm{Var}(\varepsilon)$ (d) $\hat{\mathcal{R}}_{\text{train}}$
2. As model flexibility increases, typically:
   (a) bias↑, variance↓ (b) bias↓, variance↑ (c) both ↑ (d) both ↓
3. The best predictor under squared loss is:
   (a) the median of Y (b) the mode of Y (c) $\mathbb{E}[Y \mid X]$ (d) $\mathbb{E}[Y]$
4. In 10-fold CV, each observation is used for validation:
   (a) once (b) ten times (c) never (d) a random number of times

5. If $\sigma = 12$ and $n = 36$, then $\mathrm{SE}(\bar{X})$ is:
   (a) 12 (b) 6 (c) 2 (d) 0.33
6. Which estimator of σ^2 is unbiased?
   (a) $\frac{1}{n}\sum(X_i - \bar{X})^2$ (b) $\frac{1}{n-1}\sum(X_i - \bar{X})^2$ (c) $\frac{1}{n+1}\sum(X_i - \bar{X})^2$ (d) none
7. Recall is defined as:
   (a) $\frac{TP}{TP+FP}$ (b) $\frac{TP}{TP+FN}$ (c) $\frac{TN}{TN+FP}$ (d) $\frac{TP+TN}{n}$
8. Which loss is most robust to outliers?
   (a) squared (b) absolute (c) cubic (d) exponential

Answers: 1-c, 2-b, 3-c, 4-a, 5-c, 6-b, 7-b, 8-b.

F. Practice Problems

1. In Figure 2.1, suppose $f(x) = 2x + 1$ and $\sigma = 0.5$. What is the smallest possible expected squared error for any model, and why?
2. Run the polynomial experiment with degrees 2, 5 and 12 and sketch train/test MSE against degree. At which degree does the test MSE start rising?
3. **Coding:** implement 5-fold cross-validation with `sklearn.model_selection.cross_val_score` for `LinearRegression` and report the mean and standard error of the scores.
4. Simulate sampling distributions from a *skewed* population (e.g. `rng.exponential(1, n)`) and verify with histograms that $\bar{X}$ becomes bell-shaped as n grows (CLT).

Quick Revision

Topic	One-line recall
Model	$Y = f(X) + \varepsilon$, $\mathbb{E}\varepsilon = 0$, $\mathrm{Var}\,\varepsilon = \sigma^2$.
Prediction / Inference	Accurate $\hat{Y}$ / understand effect of each X_j.
Error split	Reducible + irreducible σ^2.
Best predictors	$\mathbb{E}[Y \mid X]$ (regression); Bayes classifier $\arg\max_k P(k \mid x)$.
Risk	$\mathbb{E}[L(Y, f(X))]$ (unknown); train loss optimistic; test loss honest.
Bias–variance	$\mathbb{E}[(y_0 - \hat{f})^2] = \mathrm{Bias}^2 + \mathrm{Var} + \sigma^2$; U-shaped test error.
Hold-out / k-fold / LOOCV	1 fit / k fits (use 5 or 10) / n fits.
Classification stats	$\mathrm{Acc} = \frac{TP+TN}{n}$, $\mathrm{Prec} = \frac{TP}{TP+FP}$, $\mathrm{Rec} = \frac{TP}{TP+FN}$, F_1 harmonic mean.
Sampling distribution	Distribution of an estimator over repeated samples; spread = SE.
Sample mean	Unbiased; $\mathrm{SE} = \sigma/\sqrt{n}$; 4× data ⇒ 2× precision.
CLT	$\bar{X} \approx \mathcal{N}(\mu, \sigma^2/n)$ for large n; 95% CI $\bar{x} \pm 1.96\,\mathrm{SE}$.
Variance estimator	Divide by $n - 1$ for unbiasedness.

Chapter 3

Distance-Based Methods, Decision Trees and Naive Bayes

Learning Objectives

After studying this chapter you will be able to:

- compute **Euclidean, Manhattan, Minkowski, Chebyshev, Hamming** and **cosine** measures and explain why **feature scaling** matters;
- apply the **k-Nearest Neighbours** algorithm to classification and regression, and explain the effect of k;
- build a **decision tree** by hand using **entropy, information gain** and **Gini index**;
- explain **overfitting in trees** and **pruning**;
- state **Bayes' theorem** and apply the **Naive Bayes** classifier, including **Laplace smoothing** and the **Gaussian** variant;
- compare the three methods and choose between them;
- answer 2-, 5- and 10-mark university questions on these topics.

> *Birds of a feather flock together: the nearest neighbour of a point carries a lot of information about its class.*
>
> — Cover & Hart, 1967 (on nearest neighbours)

This chapter begins Unit II (Supervised Learning). The three methods here are among the simplest and most intuitive classifiers, and each uses a different idea:

Method	Core idea	Type (Chapter 1 / 2)
k-Nearest Neighbours	"You are like your closest neighbours"	instance-based, non-parametric
Decision tree	"Ask a sequence of yes/no questions"	model-based, non-parametric
Naive Bayes	"Use probability and Bayes' rule"	model-based, parametric

3.1 Distance-Based Methods

Many learning algorithms (k-NN, K-Means, SVMs with kernels) depend on one question: *how similar are two instances?* We answer it with a **distance** (small = similar) or a **similarity** (large = similar).

Definition: Distance metric

A function $d(\boldsymbol{a}, \boldsymbol{b})$ is a **metric** if for all points: (i) $d(\boldsymbol{a}, \boldsymbol{b}) \geq 0$ and $d(\boldsymbol{a}, \boldsymbol{b}) = 0 \iff \boldsymbol{a} = \boldsymbol{b}$; (ii) symmetry $d(\boldsymbol{a}, \boldsymbol{b}) = d(\boldsymbol{b}, \boldsymbol{a})$; (iii) triangle inequality $d(\boldsymbol{a}, \boldsymbol{c}) \leq d(\boldsymbol{a}, \boldsymbol{b}) + d(\boldsymbol{b}, \boldsymbol{c})$.

3.1.1 Common Distance Measures

For two d-dimensional points $\boldsymbol{a} = (a_1, \ldots, a_d)$ and $\boldsymbol{b} = (b_1, \ldots, b_d)$:

Measure	Formula	Use
Euclidean (L_2)	$\sqrt{\sum_i (a_i - b_i)^2}$	Default for numeric data
Manhattan (L_1)	$\sum_i \lvert a_i - b_i \rvert$	Grid-like paths; less sensitive to outliers
Minkowski (L_p)	$\left(\sum_i \lvert a_i - b_i \rvert^p\right)^{1/p}$	General family ($p = 1, 2$ above)
Chebyshev (L_∞)	$\max_i \lvert a_i - b_i \rvert$	Largest single difference
Hamming	number of positions where $a_i \neq b_i$	Binary / categorical strings
Cosine similarity	$\dfrac{\boldsymbol{a} \cdot \boldsymbol{b}}{\lVert \boldsymbol{a} \rVert \, \lVert \boldsymbol{b} \rVert}$	Text; direction, not size

Example: Computing distances

Let $\boldsymbol{a} = (1, 2, 3)$ and $\boldsymbol{b} = (4, 6, 3)$. The coordinate differences are $3, 4, 0$.

$$d_{L_2} = \sqrt{3^2 + 4^2 + 0^2} = \sqrt{25} = 5, \qquad d_{L_1} = 3 + 4 + 0 = 7,$$
$$d_{L_\infty} = \max(3, 4, 0) = 4, \qquad d_{L_3} = (27 + 64 + 0)^{1/3} = 91^{1/3} \approx 4.50.$$

Cosine similarity: $\boldsymbol{a} \cdot \boldsymbol{b} = 4 + 12 + 9 = 25$, $\lVert \boldsymbol{a} \rVert = \sqrt{14} \approx 3.742$, $\lVert \boldsymbol{b} \rVert = \sqrt{61} \approx 7.810$, so $\cos\theta = 25/29.22 \approx 0.855$.
Hamming: for the bit strings 10110 and 11100 the strings differ in positions 2 and 4, so $d_H = 2$.
Note the ordering $L_\infty \leq L_3 \leq L_2 \leq L_1$ for the same pair of points.

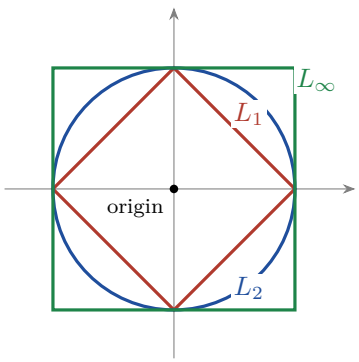


Figure 3.1. The set of all points at distance exactly 1 from the origin, for three metrics.

Reading the Figure

Each coloured outline is the set of points that are *exactly one unit away* from the origin under a different distance. Under **Euclidean** distance (L_2, blue) this set is a *circle*. Under **Manhattan** (L_1, red) it is a *diamond*, because you may only move along the axes. Under **Chebyshev** (L_∞, green) it is a *square*. The point $(1, 1)$ lies on the green square but far outside the red diamond: the *same pair of points* is counted as "close" or "far" depending on the metric. Hence the choice of metric changes which points are neighbours.

3.1.2 Feature Scaling

Distance measures add up differences across all features, so a feature with a *large numeric range* dominates the sum. We must put features on a comparable scale.

Method	Formula	Result
Min–max scaling	$x' = \dfrac{x - x_{\min}}{x_{\max} - x_{\min}}$	Values in $[0, 1]$
Standardisation (z-score)	$x' = \dfrac{x - \mu}{\sigma}$	Mean 0, standard deviation 1

Example: Why scaling matters

Three people: P = (age 25, salary 40000), Q = (45, 42000), R = (26, 80000). Raw Euclidean distances:

$$d(P, Q) = \sqrt{20^2 + 2000^2} \approx 2000.1, \qquad d(P, R) = \sqrt{1^2 + 40000^2} \approx 40000.0.$$

The salary difference swamps age, so Q (20 years older) looks far closer to P than R (1 year older) does, which is silly. Now min–max scale with age range 20–60 and salary range 20 000–100 000:

$$P' = (0.125,\ 0.250), \quad Q' = (0.625,\ 0.275), \quad R' = (0.150,\ 0.750).$$

$d(P', Q') = \sqrt{0.5^2 + 0.025^2} \approx 0.5006$ and $d(P', R') = \sqrt{0.025^2 + 0.5^2} \approx 0.5006$: after scaling, age and salary contribute fairly.

Common Mistake

Always compute the scaling parameters ($x_{\min}, x_{\max}, \mu, \sigma$) on the **training set only** and then apply the *same* numbers to the test set. Using the test data to compute them leaks information.

3.2 Nearest Neighbours (k-NN)

3.2.1 The Algorithm

k-NN is the simplest learning algorithm: it has **no training phase**. It just stores the training data (a *lazy learner*) and does all the work at prediction time.

Definition: k-Nearest Neighbours

To predict for a new point $\boldsymbol{x}_q$:

1. Compute the distance from $\boldsymbol{x}_q$ to every training point.
2. Pick the k training points with the smallest distances (the *neighbourhood* $N_k(\boldsymbol{x}_q)$).
3. **Classification:** output the *majority class* among the k neighbours.
   Regression: output the *mean* of their target values.

Example: k-NN classification by hand

Training data (two features, two classes) and the query $\boldsymbol{x}_q = (5, 4)$:

Point	x_1	x_2	Class	Distance to $(5,4)$	Rank
P_4	6	5	B	$\sqrt{1+1} = 1.414$	1
P_5	7	4	B	$\sqrt{4+0} = 2.000$	2–3
P_6	5	6	B	$\sqrt{0+4} = 2.000$	2–3
P_7	4	2	A	$\sqrt{1+4} = 2.236$	4
P_3	3	2	A	$\sqrt{4+4} = 2.828$	5
P_2	2	3	A	$\sqrt{9+1} = 3.162$	6
P_1	1	1	A	$\sqrt{16+9} = 5.000$	7

k	Neighbours	Prediction
1	P_4 (B)	**B**
3	P_4, P_5, P_6 = B,B,B	**B**
5	B,B,B,A,A (3 vs 2)	**B**
7	all points: 4 A vs 3 B	**A**

The prediction *changes* from B to A when k grows to 7, which shows how sensitive k-NN is to k.

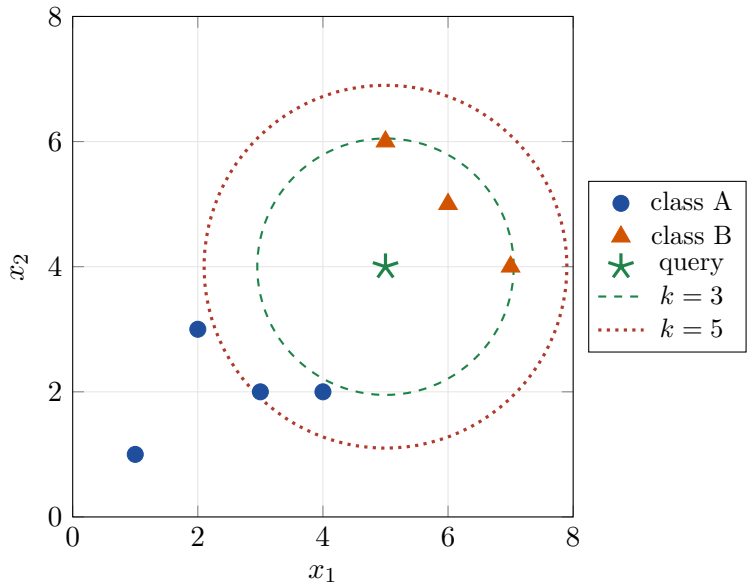


Figure 3.2. Neighbourhoods of the query point for $k = 3$ and $k = 5$.

Reading the Figure

The **green star** is the query point. Blue circles are class A and orange triangles are class B. The **dashed green circle** is just big enough to contain the 3 nearest points: all three are orange, so $k = 3$ votes B. The **dotted red circle** is large enough to contain 5 points: three orange and two blue, so $k = 5$ still votes B, but by a smaller margin. If we kept enlarging the circle to include all 7 points the blue majority would win. The *radius adapts* to the local density of data; this is why k-NN is called *local*.

3.2.2 Choosing k

	Small k (e.g. 1)	**Large k**
Decision boundary	Very jagged, follows every point	Smooth
Bias	Low	High
Variance	High (sensitive to noise)	Low
Risk	Overfitting	Underfitting

k is a **hyperparameter**, chosen by cross-validation (Chapter 2). Use an *odd* k for two-class problems to avoid ties. A common starting point is $k \approx \sqrt{n}$.

Key Idea

$1/k$ is a measure of model flexibility: $k = 1$ is the *most flexible* (and most variable) k-NN; $k = n$ always predicts the overall majority class (the least flexible). This is the bias–variance trade-off of Chapter 2 in action.

3.2.3 Weighted k-NN and k-NN Regression

In **weighted k-NN**, closer neighbours get more influence, with a weight $w_i = 1/d_i$ (or $1/d_i^2$).

Example: k-NN regression

Use the house data of Chapter 1: size $x = 1, 2, 3, 4, 5$ with price $y = 15, 26, 29, 44, 46$. Predict the price at $x_q = 3.4$ with $k = 2$. The two nearest sizes are $x = 3$ (distance 0.4, price 29) and $x = 4$ (distance 0.6, price 44).

- **Plain average:** $\hat{y} = (29 + 44)/2 = 36.5$.
- **Weighted ($w = 1/d$):** $w_1 = 2.5$, $w_2 = 1.667$; $\hat{y} = \dfrac{2.5(29) + 1.667(44)}{2.5 + 1.667} = \dfrac{145.83}{4.167} = 35.0$.

The weighted answer is pulled towards the closer house (price 29). For comparison, the linear model $8 + 8x$ from Chapter 1 predicts 35.2.

3.2.4 Strengths and Weaknesses

Advantages	Disadvantages
Very simple; no training	Slow prediction: must scan all n points ($O(nd)$ per query)
Naturally handles multiple classes	Needs a lot of memory (stores everything)
No assumption about the data shape	Sensitive to feature scale and irrelevant features
Improves as more data arrives	Suffers from the *curse of dimensionality*

Curse of dimensionality. In high dimensions, points become almost equally far apart, so the idea of a "nearest" neighbour loses meaning and k-NN degrades. Remedies: feature selection and dimensionality reduction such as PCA (Chapter 8).

3.3 Decision Trees

3.3.1 Idea and Structure

A decision tree predicts by asking a sequence of simple questions about the features, like a game of "twenty questions".

Definition: Decision tree

A **decision tree** is a flowchart-like model in which each **internal node** tests one feature, each **branch** is an outcome of the test, and each **leaf** holds a prediction (a class label for *classification trees*, or a number for *regression trees*). The top node is the **root**.

The learning problem is: *which feature should we test at each node?* We choose the test that makes the resulting groups as **pure** as possible, i.e. each group dominated by one class.

3.3.2 Measures of Impurity

Let a node contain examples of C classes with proportions $p_1, \dots, p_C$.

Definition: Entropy

$$H = -\sum_{i=1}^{C} p_i \log_2 p_i \qquad (0 \log 0 := 0).$$

Entropy is 0 when the node is pure and is maximal ($\log_2 C$) when all classes are equally likely. It is measured in *bits*.

Definition: Gini index

$$G = 1 - \sum_{i=1}^{C} p_i^2.$$

Gini is 0 for a pure node and equals 0.5 for a 50/50 two-class node. It is the probability that two items drawn at random from the node have different classes.

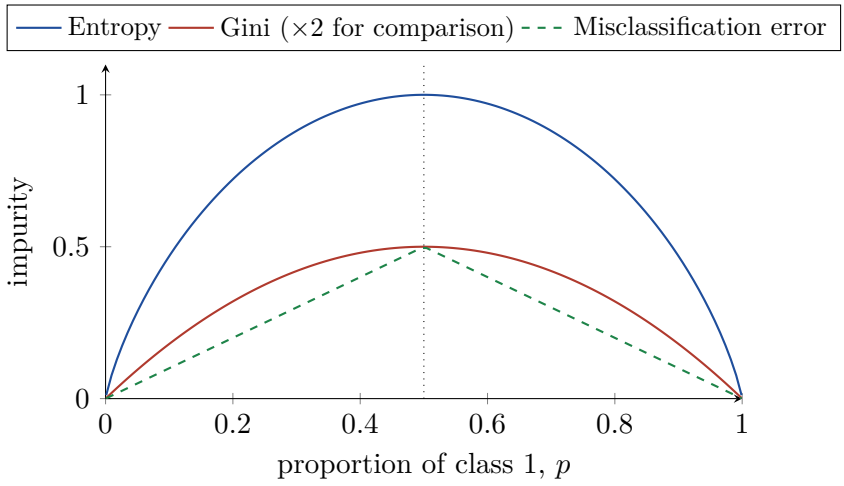


Figure 3.3. Impurity measures for a two-class node as a function of p.

Reading the Figure

The horizontal axis is the **proportion of class 1** in a node: $p = 0$ or $p = 1$ means a *pure* node and $p = 0.5$ is the most mixed. All three curves are **zero at the ends** (pure) and **highest at the middle** (maximum confusion). **Entropy** (blue) is slightly more curved than **Gini** (red; drawn here as $2p(1-p)$ so that it peaks at 1 like entropy). In practice they almost always choose the same splits. The dashed green *misclassification error* has a sharp corner and is less sensitive to changes, which is why it is rarely used for growing trees.

Definition: Information gain

If a split on feature A divides a set S into subsets S_v (one for each value v of A), then

$$\mathrm{Gain}(S, A) = H(S) - \sum_{v} \frac{|S_v|}{|S|} H(S_v).$$

It is the *reduction in entropy* achieved by the split. At each node the tree picks the feature with the **highest information gain**. (The ID3 algorithm does exactly this; C4.5 uses the *gain ratio* to avoid favouring features with many values; CART uses the Gini index and binary splits.)

3.3.3 Worked Example: Building a Tree with ID3

We want to decide whether to **play tennis** from the weather. There are 14 past days.

Day	Outlook	Temp.	Humidity	Wind	Play
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Step 1: entropy of the whole set. There are 9 Yes and 5 No:

$$H(S) = -\tfrac{9}{14}\log_2\tfrac{9}{14} - \tfrac{5}{14}\log_2\tfrac{5}{14} = 0.410 + 0.530 = \mathbf{0.940}.$$

Step 2: information gain of every feature.

Split	Values (Yes, No)	Entropies	Gain
Outlook	Sunny (2,3), Overcast (4,0), Rain (3,2)	0.971, 0, 0.971	**0.247**
Humidity	High (3,4), Normal (6,1)	0.985, 0.592	0.152
Wind	Weak (6,2), Strong (3,3)	0.811, 1.000	0.048
Temperature	Hot (2,2), Mild (4,2), Cool (3,1)	1.000, 0.918, 0.811	0.029

Detail for Outlook:

$$\text{Gain} = 0.940 - \tfrac{5}{14}(0.971) - \tfrac{4}{14}(0) - \tfrac{5}{14}(0.971) = 0.940 - 0.694 = 0.247.$$

Outlook has the highest gain, so it becomes the root.

Step 3: recurse on each branch.

- **Overcast:** all 4 days are Yes ⇒ pure ⇒ leaf *Yes*.
- **Sunny** (D1, D2, D8, D9, D11; 2 Yes, 3 No): splitting on Humidity gives High → {D1, D2, D8} all No, Normal → {D9, D11} all Yes. Both pure, so Humidity is chosen.
- **Rain** (D4, D5, D6, D10, D14; 3 Yes, 2 No): splitting on Wind gives Weak → {D4, D5, D10} all Yes, Strong → {D6, D14} all No.

Reading the Figure

Blue boxes are decision nodes (questions), **green** and **red** boxes are leaves (final answers: Yes/No). To classify a new day, start at the *root* and follow the branch matching the day's value at each question until you reach a leaf. For example, a day that is *Rain* and *Weak wind*: Outlook = Rain → Wind? → Weak → **Yes**. The tree also reads as rules: *IF Outlook=Overcast THEN Play=Yes; IF Outlook=Sunny AND Humidity=High THEN No;* ... This is why trees are so easy to *interpret*.

The same split using the Gini index. $G(S) = 1 - (\frac{9}{14})^2 - (\frac{5}{14})^2 = 0.459$. For Outlook, $G(\text{Sunny}) = G(\text{Rain}) = 1 - (0.4)^2 - (0.6)^2 = 0.48$ and $G(\text{Overcast}) = 0$, so the weighted Gini is $\frac{5}{14}(0.48) + \frac{4}{14}(0) + \frac{5}{14}(0.48) = 0.343$ and the Gini gain is $0.459 - 0.343 = 0.116$, again the best feature.

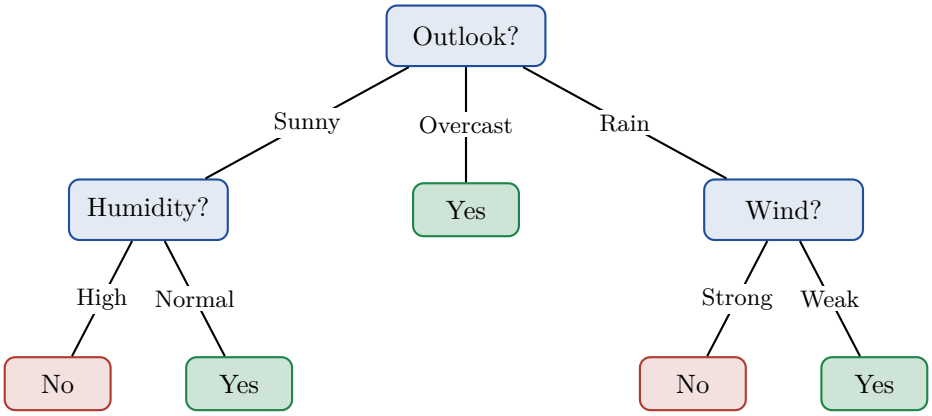


Figure 3.4. The decision tree learned by ID3 from the Play-Tennis data.

3.3.4 Algorithm Summary

1. Start with all training data at the root.
2. If the node is pure (or a stopping rule applies), make it a leaf labelled with the majority class.
3. Otherwise, evaluate every candidate split; choose the one with the highest gain (or largest drop in Gini).
4. Split the data and **recurse** on each child.

Continuous features. Sort the values, try thresholds halfway between consecutive values (e.g. "Age ≤ 30?"), and choose the threshold with the best gain.

Regression trees. Leaves store the *mean* target of their training points; the best split minimises the sum of squared errors (variance) in the two children.

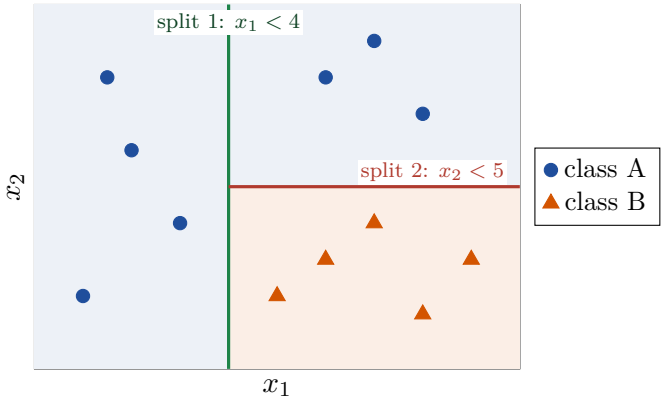


Figure 3.5. A decision tree with two splits carves the feature space into axis-parallel rectangles.

Reading the Figure

Every split in a tree tests *one feature against a threshold*, which draws a **straight line parallel to an axis**. The **green vertical line** is the root test $x_1 < 4$: everything to its left is class A. On the right, the **red horizontal line** tests $x_2 < 5$: below it is class B (orange), above it class A. The final result is three *rectangular regions*, each assigned one class. Trees therefore produce *boxy* decision boundaries, which suit data with axis-aligned

structure but approximate diagonal boundaries only with many small steps.

3.3.5 Overfitting and Pruning

A tree allowed to grow until every leaf is pure will memorise the training data, noise included (low bias, **high variance**; see Chapter 2). We control complexity by **pruning**.

Pre-pruning (early stopping)	Stop growing early: limit the depth (`max_depth`), require a minimum number of samples to split (`min_samples_split`) or in a leaf (`min_samples_leaf`), or require a minimum gain.
Post-pruning	Grow the full tree, then *cut back* branches that do not improve accuracy on a validation set (reduced-error pruning) or minimise cost $=$ error $+ \alpha \cdot$ (number of leaves) (cost-complexity pruning).

3.3.6 Strengths and Weaknesses

Advantages	Disadvantages
Easy to understand and visualise; human-readable rules	Prone to overfitting
Handles numeric and categorical features	Unstable: small data changes can give a very different tree
No feature scaling needed	Axis-parallel (boxy) boundaries only
Fast prediction; performs feature selection implicitly	Biased towards features with many values

The instability of single trees is fixed by combining many of them into *ensembles* such as Random Forests (Chapter 5).

3.4 Naive Bayes

3.4.1 Bayes' Theorem

Definition: Bayes' theorem

For a class C and observed evidence $\boldsymbol{x}$,

$$P(C \mid \boldsymbol{x}) = \frac{P(\boldsymbol{x} \mid C)\, P(C)}{P(\boldsymbol{x})}, \qquad P(\boldsymbol{x}) = \sum_{c} P(\boldsymbol{x} \mid c) P(c).$$

Here $P(C \mid \boldsymbol{x})$ is the **posterior**, $P(\boldsymbol{x} \mid C)$ the **likelihood**, $P(C)$ the **prior** and $P(\boldsymbol{x})$ the **evidence**.

It follows directly from the definition of conditional probability, $P(C \mid \boldsymbol{x})P(\boldsymbol{x}) = P(\boldsymbol{x}, C) = P(\boldsymbol{x} \mid C)P(C)$. In words: *posterior* (what we believe after seeing the evidence) $\propto$ *likelihood* (how well the class explains the evidence) $\times$ *prior* (what we believed before).

Example: Spam and the word "offer"

Suppose 30% of all e-mail is spam, so $P(\text{spam}) = 0.3$ and $P(\text{ham}) = 0.7$. The word "offer" occurs in 60% of spam, $P(\text{offer} \mid \text{spam}) = 0.6$, but only in 10% of ham, $P(\text{offer} \mid \text{ham}) = 0.1$.

An e-mail contains “offer”. How likely is it to be spam?

$$P(\text{spam} \mid \text{offer}) = \frac{0.6 \times 0.3}{0.6 \times 0.3 + 0.1 \times 0.7} = \frac{0.18}{0.18 + 0.07} = \frac{0.18}{0.25} = \mathbf{0.72}.$$

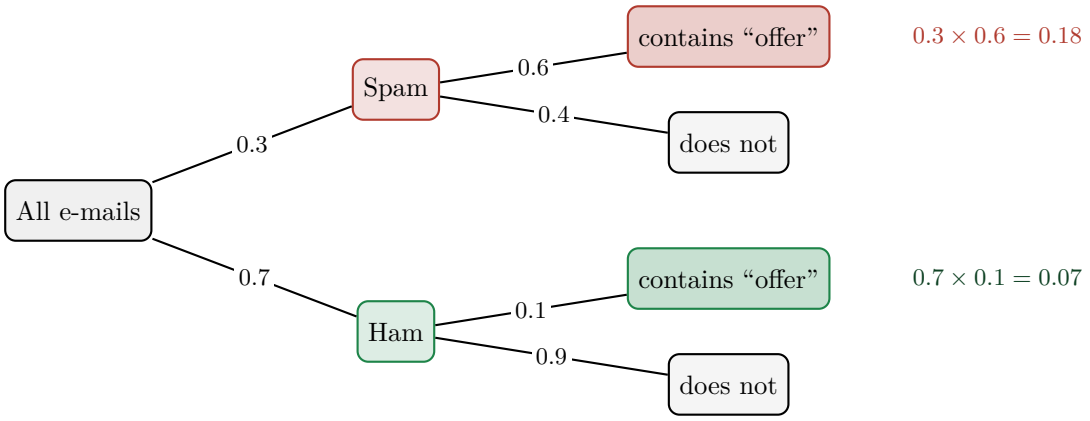


Figure 3.6. Probability tree for the spam example. Only the two “offer” leaves matter once we know the word appeared.

Reading the Figure

Read the tree **left to right**, *multiplying probabilities along a path.* First the e-mail is spam (0.3) or ham (0.7). Then, given its class, it contains “offer” with probability 0.6 (spam) or 0.1 (ham). The **two paths that end in “contains offer”** have joint probabilities 0.18 (spam) and 0.07 (ham). Because we *know* the word appeared, we only look at those two paths, and the posterior is the spam share of their total: $0.18/(0.18 + 0.07) = 0.72$. The prior of 30% has been **updated** to 72% by the evidence.

3.4.2 The Naive Bayes Classifier

With many features $\boldsymbol{x} = (x_1, \ldots, x_d)$, the likelihood $P(x_1, \ldots, x_d \mid C)$ would need an enormous number of probabilities. Naive Bayes makes one simplifying assumption.

Definition: Naive (conditional independence) assumption

Given the class, the features are **independent**:

$$P(x_1, \ldots, x_d \mid C) = \prod_{j=1}^{d} P(x_j \mid C).$$

The classification rule becomes (the denominator $P(\boldsymbol{x})$ is the same for every class, so it can be dropped):

$$\hat{y} = \arg\max_{c} \; P(c) \prod_{j=1}^{d} P(x_j \mid c). \tag{3.1}$$

The assumption is usually *false* (in e-mail, “free” and “offer” tend to appear together), hence “naive”, yet the classifier works surprisingly well, because we only need the class with the *largest* score, not exact probabilities.

Training = counting. Estimate $P(c) = N_c/N$ and $P(x_j = v \mid c) = \dfrac{\text{count}(x_j = v,\ \text{class } c)}{N_c}$ from the training data.

3.4.3 Worked Example: Play-Tennis Again

Classify the new day $\boldsymbol{x} = (\text{Sunny},\ \text{Cool},\ \text{High},\ \text{Strong})$ using the 14-day table.

Priors: $P(\text{Yes}) = \frac{9}{14}$, $P(\text{No}) = \frac{5}{14}$.

Likelihood	Given Yes (9 days)	Given No (5 days)
$P(\text{Sunny} \mid \cdot)$	2/9	3/5
$P(\text{Cool} \mid \cdot)$	3/9	1/5
$P(\text{High} \mid \cdot)$	3/9	4/5
$P(\text{Strong} \mid \cdot)$	3/9	3/5

$$\text{score}(\text{Yes}) = \tfrac{9}{14} \cdot \tfrac{2}{9} \cdot \tfrac{3}{9} \cdot \tfrac{3}{9} \cdot \tfrac{3}{9} = 0.00529, \qquad \text{score}(\text{No}) = \tfrac{5}{14} \cdot \tfrac{3}{5} \cdot \tfrac{1}{5} \cdot \tfrac{4}{5} \cdot \tfrac{3}{5} = 0.02057.$$

Decision: $0.02057 > 0.00529 \Rightarrow$ predict **No**.

Posterior probabilities (normalise): $P(\text{No} \mid \boldsymbol{x}) = \dfrac{0.02057}{0.02057 + 0.00529} = 0.795$ and $P(\text{Yes} \mid \boldsymbol{x}) = 0.205$.

3.4.4 Laplace Smoothing

Common Mistake

The zero-frequency problem. If a value never occurs with a class in training (e.g. "Overcast" never occurs with "No"), then $P(\text{Overcast} \mid \text{No}) = 0$ and the *whole product becomes zero*, whatever the other features say.

Laplace (add-one) smoothing adds a pseudo-count α (usually 1) to every count:

$$P(x_j = v \mid c) = \frac{\text{count}(x_j = v, c) + \alpha}{N_c + \alpha V_j},$$

where V_j is the number of different values of feature j.

Example: Smoothing

$\text{count}(\text{Overcast}, \text{No}) = 0$, $N_{\text{No}} = 5$, and Outlook has $V = 3$ values. With $\alpha = 1$: $P(\text{Overcast} \mid \text{No}) = \dfrac{0+1}{5+3} = \dfrac{1}{8} = 0.125$ instead of 0.

3.4.5 Variants of Naive Bayes

Variant	Feature type	Model for $P(x_j \mid c)$
Gaussian	Continuous	Normal density with class-wise mean μ_{jc} and variance σ^2_{jc}
Multinomial	Counts (word counts)	Relative frequency of word in class (with smoothing)
Bernoulli	Binary (word present/absent)	Probability of presence/absence

For the **Gaussian** variant,

$$P(x_j \mid c) = \frac{1}{\sqrt{2\pi}\,\sigma_{jc}} \exp\!\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right).$$

Example: Gaussian Naive Bayes with one feature

Heights: males have $\mu = 175$, $\sigma = 7$ cm; females have $\mu = 162$, $\sigma = 6$ cm; priors are equal. Classify a person of height 170 cm.

$$P(170 \mid \text{male}) = \tfrac{1}{7\sqrt{2\pi}} e^{-25/98} = 0.0570 \times 0.775 = 0.0442,$$
$$P(170 \mid \text{female}) = \tfrac{1}{6\sqrt{2\pi}} e^{-64/72} = 0.0665 \times 0.411 = 0.0273.$$

Since the priors are equal, the larger likelihood wins: **male**, with posterior $0.0442/(0.0442 + 0.0273) \approx 0.62$. (Only 62% confident because 170 is plausible for both.)

Key Idea

Numerical tip. Multiplying many small probabilities underflows to 0. Real implementations add **logarithms** instead: $\hat{y} = \arg\max_c \left[\ln P(c) + \sum_j \ln P(x_j \mid c)\right]$.

Advantages	Disadvantages
Very fast to train and predict (just counting)	Independence assumption rarely true
Works well with little data and many features	Probability estimates can be poorly calibrated
Excellent baseline for text (spam, sentiment)	Zero-frequency problem (needs smoothing)
Handles multiple classes naturally	Cannot learn interactions between features

3.5 Putting It Together: The Three Methods in Python

Try It Yourself

Compare all three on the classic *Iris* flower dataset (150 flowers, 4 measurements, 3 species).

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split,
    cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB

X, y = load_iris(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

models = {
  "k-NN (k=5)": make_pipeline(StandardScaler(),
```

```
                                    KNeighborsClassifier(n_neighbors=5)
                                        ),
    "Decision tree": DecisionTreeClassifier(max_depth=3,
        random_state=0),
    "Naive Bayes": GaussianNB(),
}
for name, m in models.items():
    m.fit(Xtr, ytr)
    cv = cross_val_score(m, X, y, cv=5).mean()
    print(f"{name:14s} test = {m.score(Xte, yte):.3f}  CV = {cv
        :.3f}")
```

Output:

Model	Test accuracy	5-fold CV accuracy
k-NN ($k = 5$)	0.911	0.960
Decision tree (depth 3)	0.933	0.960
Naive Bayes	0.911	0.953

All three do well here. The single test split (45 flowers) is noisy, so the *cross-validated* numbers are more reliable and show the methods are practically tied.

Effect of k (same data, scaled):

k	Train acc.	Test acc.	Comment
1	1.000	0.933	memorises training data
3	0.971	0.911	
5	0.981	0.911	
15	0.981	0.933	smooth boundary
51	0.876	0.822	**underfits**: both fall

A huge k averages over too much of the data and accuracy drops on *both* sets: underfitting, exactly as predicted.

A tiny tree you can read (`max_depth=2`, printed with `export_text`):

```
|--- petal_wid <= 0.70              -> class 0 (setosa)
|--- petal_wid >  0.70
|   |--- petal_wid <= 1.55         -> class 1 (versicolor)
|   |--- petal_wid >  1.55         -> class 2 (virginica)
```

Just *two questions about petal width* separate the three species.

3.6 Comparison of the Three Methods

	k-NN	**Decision tree**	**Naive Bayes**
Learner type	Lazy, instance-based	Eager, model-based	Eager, model-based
Training cost	None	Moderate	Very low
Prediction cost	High	Very low	Very low
Assumption	Similar points share classes	Axis-aligned splits	Features independent given class
Needs scaling?	**Yes**	No	No (Gaussian: no)
Interpretable?	Low	**High**	Moderate
Main hyperparameter	k, metric	depth, min samples	smoothing α
Typical weakness	Slow; high dimensions	Overfits; unstable	Independence assumption

Chapter Summary

- Distance measures (L_1, L_2, L_p, L_∞, Hamming, cosine) define similarity; **scale features** first (min–max or z-score, fitted on training data only).
- **k-NN** stores data and predicts by majority vote (classification) or mean (regression) of the k closest points. Small $k \Rightarrow$ high variance; large $k \Rightarrow$ high bias.
- **Decision trees** split on the feature with highest **information gain** (entropy) or largest Gini reduction. Full trees overfit; control with **pruning**.
- Entropy $H = -\sum p_i \log_2 p_i$; Gini $= 1 - \sum p_i^2$; Gain $= H(S) - \sum \frac{|S_v|}{|S|} H(S_v)$.
- **Naive Bayes**: $\hat{y} = \arg\max_c P(c) \prod_j P(x_j \mid c)$, assuming conditional independence; use **Laplace smoothing** and log-probabilities; variants: Gaussian, multinomial, Bernoulli.
- In the Play-Tennis example the tree root is Outlook (gain 0.247) and Naive Bayes predicts "No" with posterior 0.795 for (Sunny, Cool, High, Strong).

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define Euclidean and Manhattan distance. **[2 marks]**
2. Why is feature scaling necessary for k-NN? **[2 marks]**
3. Why is k-NN called a lazy learner? **[2 marks]**
4. What happens when $k = 1$ and when $k = n$ in k-NN? **[2 marks]**
5. Define entropy and Gini index. **[2 marks]**
6. What is information gain? **[2 marks]**
7. What is pruning? Name its two types. **[2 marks]**
8. State Bayes' theorem. **[2 marks]**
9. Why is Naive Bayes called "naive"? **[2 marks]**
10. What is Laplace smoothing and why is it needed? **[2 marks]**

Model Answer: Q4 and Q10

Q4. For $k = 1$ the prediction is the label of the single nearest point: the boundary is very jagged, bias is low, variance is high (overfitting). For $k = n$ every query gets the overall majority class: the model is a constant, bias is high and variance is zero (underfitting).
Q10. Laplace smoothing adds a pseudo-count (usually 1) to every feature-value count so that $P(x_j \mid c) = \frac{\text{count}+1}{N_c+V_j}$. It is needed to avoid zero probabilities for unseen feature–class combinations, which would force the whole product $\prod P(x_j \mid c)$ to zero.

B. Short Answer Questions (5 marks each)

1. Explain the k-NN algorithm for classification with an example. **[5 marks]**
2. Discuss the effect of the value of k on k-NN (bias, variance, over/underfitting). **[5 marks]**
3. List the advantages and disadvantages of k-NN. What is the curse of dimensionality? **[5 marks]**
4. Explain the structure of a decision tree. Differentiate entropy and Gini index. **[5 marks]**
5. Explain overfitting in decision trees. How can it be reduced? **[5 marks]**
6. Explain the Naive Bayes classifier and its independence assumption. **[5 marks]**
7. Compare k-NN, decision trees and Naive Bayes in a table. **[5 marks]**

Model Answer: B4 (outline for 5 marks)

Structure: root node (first test), internal nodes (feature tests), branches (outcomes), leaves (class label or value); classification of a new point follows the path from root to a leaf. *Entropy* $H = -\sum p_i \log_2 p_i$: measures disorder of class labels; 0 for a pure node, 1 for a 50/50 binary node. *Gini* $= 1 - \sum p_i^2$: probability of mis-classifying a random item; 0 for pure, 0.5 for 50/50.

	Entropy	**Gini**
Formula	$-\sum p_i \log_2 p_i$	$1 - \sum p_i^2$
Range (2 classes)	0 to 1	0 to 0.5
Cost	Needs logarithm	Cheaper (no log)
Used in	ID3, C4.5	CART

C. Long Answer Questions (10 marks each)

1. Explain the ID3 algorithm. Using the 14-day Play-Tennis table, compute the information gain of *Outlook* and show that it is chosen as the root. **[10 marks]**
2. Explain the Naive Bayes classifier with Laplace smoothing. Use the Play-Tennis data to classify (Sunny, Cool, High, Strong). **[10 marks]**
3. Describe the k-NN algorithm for classification and regression. Illustrate with a numerical example and discuss the choice of k and distance metric. **[10 marks]**
4. Explain distance measures used in machine learning with examples. Why must features be scaled? **[10 marks]**
5. Discuss pre-pruning and post-pruning in decision trees. How do trees compare to k-NN and Naive Bayes in terms of interpretability and cost? **[10 marks]**

Model Answer: C1 (key steps)

1. $H(S) = 0.940$ (9 Yes, 5 No). 2. Outlook: Sunny (2Y,3N) $H = 0.971$; Overcast (4Y,0N) $H = 0$; Rain (3Y,2N) $H = 0.971$. 3. Gain $= 0.940 - \frac{5}{14}(0.971) - \frac{4}{14}(0) - \frac{5}{14}(0.971) = 0.247$. 4. Gains of the others: Humidity 0.152, Wind 0.048, Temperature 0.029. Outlook is largest, so it is the root. 5. Overcast is pure (Yes); Sunny splits on Humidity; Rain splits on Wind. Draw the tree of Figure 3.4.

D. Numerical Problems

1. Find the Euclidean and Manhattan distance between $(2, 3)$ and $(6, 6)$. *(Answer:* 5 *and* 7*.)*
2. Training points: $(1, 2)$ A, $(2, 1)$ A, $(3, 3)$ A, $(4, 5)$ B, $(5, 4)$ B. Classify the query $(4, 4)$ using $k = 3$ and $k = 5$ (Euclidean). *(Distances:* $1, 1, 1.414, 3.606, 3.606$*. Answer:* $k = 3$*: B,B,A* $\rightarrow$ ***B****;* $k = 5$*: 2B vs 3A* $\rightarrow$ ***A****.)*
3. Distances to a query are 1 and 2 with target values 10 and 20. Give the weighted $(1/d)$ k-NN regression value for $k = 2$. *(Answer:* $\frac{1 \cdot 10 + 0.5 \cdot 20}{1.5} = 13.33$*.)*
4. Compute the entropy of a node with 6 positive and 2 negative examples. *(Answer:* $-0.75 \log_2 0.75 - 0.25 \log_2 0.25 = 0.811$*.)*
5. A node with 4 positives and 4 negatives is split into $(3+, 1-)$ and $(1+, 3-)$. Compute the information gain. *(Answer:* $1 - \frac{4}{8}(0.811) - \frac{4}{8}(0.811) = 0.189$*.)*
6. Compute the Gini index of a node with 3 "yes" and 1 "no". *(Answer:* $1 - 0.5625 - 0.0625 = 0.375$*.)*
7. $P(\text{spam}) = 0.4$, $P(\text{"free"} \mid \text{spam}) = 0.5$, $P(\text{"free"} \mid \text{ham}) = 0.05$. Find $P(\text{spam} \mid \text{"free"})$. *(Answer:* $\frac{0.4 \cdot 0.5}{0.4 \cdot 0.5 + 0.6 \cdot 0.05} = \frac{0.20}{0.23} = 0.870$*.)*
8. A word never appears in the 100 words of class "ham"; vocabulary size is 50. Find its smoothed probability. *(Answer:* $\frac{0+1}{100+50} = 0.0067$*.)*

E. Multiple-Choice Questions

1. k-NN does its main computation during:
   (a) training (b) prediction (c) pruning (d) scaling
2. Which distance is the sum of absolute coordinate differences?
   (a) Euclidean (b) Chebyshev (c) Manhattan (d) Cosine
3. The entropy of a pure node is:
   (a) 1 (b) 0.5 (c) 0 (d) ∞
4. A decision tree chooses the split with the:
   (a) lowest gain (b) highest information gain (c) most branches (d) fewest samples
5. Which method needs feature scaling the most?
   (a) Decision tree (b) Naive Bayes (c) k-NN (d) none
6. Overfitting of a decision tree is reduced by:
   (a) growing it deeper (b) pruning (c) removing the root (d) adding duplicate data
7. In Naive Bayes the features are assumed to be:
   (a) identical (b) correlated (c) conditionally independent given the class (d) Gaussian only
8. Laplace smoothing prevents:
   (a) overfitting of trees (b) zero probabilities (c) scaling errors (d) ties in k-NN

9. The Gini index of a 50/50 two-class node is:
   (a) 1 (b) 0.5 (c) 0.25 (d) 0
10. A very large k in k-NN typically causes:
    (a) overfitting (b) underfitting (c) zero error (d) no change

Answers: 1-b, 2-c, 3-c, 4-b, 5-c, 6-b, 7-c, 8-b, 9-b, 10-b.

F. Practice Problems

1. Using the Play-Tennis table, find the best feature to split on inside the *Sunny* branch using information gain, and verify that Humidity gives gain 0.971.
2. Classify (Overcast, Mild, High, Weak) with Naive Bayes using add-one smoothing.
3. **Coding:** using `sklearn`, plot training and test accuracy of k-NN on Iris for $k = 1, 3, \ldots, 31$ and identify the best k by 5-fold cross-validation.
4. **Coding:** fit `DecisionTreeClassifier` with `max_depth` from 1 to 10 and plot train vs. test accuracy. At which depth does overfitting begin?

Quick Revision

Topic	One-line recall
Euclidean / Manhattan	$\sqrt{\sum (a_i - b_i)^2}$ / $\sum \lvert a_i - b_i \rvert$.
Scaling	Min–max $\frac{x - x_{\min}}{x_{\max} - x_{\min}}$; z-score $\frac{x - \mu}{\sigma}$; fit on train only.
k-NN	Lazy; majority vote / mean of k closest; small k overfits, large k underfits.
Weighted k-NN	$w_i = 1/d_i$; closer neighbours count more.
Entropy / Gini	$-\sum p \log_2 p$ / $1 - \sum p^2$; both 0 if pure.
Information gain	$H(S) - \sum \frac{\lvert S_v \rvert}{\lvert S \rvert} H(S_v)$; choose the maximum.
Pruning	Pre (max depth, min samples) / post (cut back, cost-complexity).
Tree boundary	Axis-parallel rectangles.
Bayes' theorem	Posterior = likelihood × prior / evidence.
Naive Bayes	$\arg\max_c P(c) \prod_j P(x_j \mid c)$; conditional independence; Laplace $\frac{n+1}{N_c + V}$.
Play-Tennis results	Root Outlook (gain 0.247); NB for (S,C,H,Str): No, $P = 0.795$.